CAST: 리전 용량 제약을 고려한 탄소 인지 시공간 워크로드 스케줄링

CAST: Carbon-Aware Spatio-Temporal Scheduling under Capacity Constraints

초록 데이터센터 탄소 배출량은 실행 리전과 시각에 따라 크게 달라진다. 그런데 탄소 예측이 실측에 가까워질수록 미룰 수 있는 작업이 같은 깨끗한 시각과 리전으로 몰려 리전 용량을 넘긴다. 작업이 시작할 때만 용량을 확인하는 2단계 설계에서 LSTM 예측을 완전예지로 바꾸면 상한 위반이 18건에서 152건으로 늘고, 용량을 두지 않은 선행 정책을 재현하면 작업의 99.75%가 한 리전으로 몰린다. 상한이 걸리는 곳은 리전별 상한 배치를 바꿔도 공간 이동이 모이는 캘리포니아와 프랑스로, 시간 이동이 값어치를 내는 리전과 다르다. 작업이 고른 시각에 자리가 있는지를 시작할 때만 확인하는 대신, 이미 정한 배정 전부와 함께 용량을 보면 몰린 작업은 상한에서 멈추고 나머지는 다음으로 깨끗한 자리로 간다. 본 연구가 제안하는 CAST는 여기에 더해 대기 중인 작업 전부를 매 슬롯 한 판에 다시 배정해, 다시 풀지 않는 설계보다 절감을 8.07%포인트 더한다. 이를 위해 LSTM 예측 위에서 로드밸런서와 스케줄러가 모든 대기 작업이 함께 보는 용량 제약 아래 리전과 시작 시각을 선형계획으로 다시 정하고(분수해가 난 슬롯만 정수 계획), 상한 초과를 목적의 첫 항으로 둔다. 탄소와 지연의 가중치는 슬롯마다 파레토 무릎점에서 자동으로 고르며, 무릎점은 사람이 가중치를 고르지 않아도 합성 워크로드에서 전선 바로 안쪽에 머물고 같은 지연의 고정 가중치보다 작업을 덜 기다리게 한다. 8개 리전 1년치 실측 탄소집약도와 합성 작업 143,200건에서 CAST는 예측기에 따라 배출을 64.44~71.93%, 탄소만 볼 때 95.59~95.87% 줄였고 마감과 상한 위반은 없었으며 예약 자리를 두지 않아도 같았다. 작업이 길고 작업시간의 절반이 미룰 수 없는 Azure VM 트레이스에서도 위반은 없었고, 용량을 지키는 비용은 16.56%포인트로 합성 워크로드의 열 배를 넘었다.

주제어: 탄소 인지 스케줄링, 워크로드 이동, 리전 용량, 선형 계획법, 롤링 지평, LSTM

Abstract The carbon footprint of a cloud workload depends on where and when it runs. Yet the closer the carbon forecast comes to the truth, the more deferrable jobs crowd into the same clean hour and region and overrun its capacity: in a two-stage design that checks capacity only when each job starts, replacing the LSTM forecast with a perfect one raises ceiling violations from 18 to 152, and a reproduced capacity-free policy sends 99.75% of jobs to a single region. Capacity binds where spatial shifting converges, in California and France, under every regional allocation of the ceiling we tried, and these are not the regions where temporal shifting pays. Checking capacity against all assignments already made, rather than only when a job starts, makes crowded jobs stop at the ceiling while the rest move to the next cleanest place. We present CAST, which in addition re-assigns all waiting jobs together in one solve at every slot, adding 8.07 percentage points of savings over a design that never re-solves. To do so, on top of an LSTM forecaster, a load balancer and a scheduler re-decide every waiting job's region and start slot under a capacity constraint shared by all of them, by linear programming at each slot (integer programming only where the relaxation is fractional), and minimise overflow before anything else. The carbon-latency weight is chosen automatically at each slot's Pareto knee, which, without a hand-set weight, stays just inside the frontier on the synthetic workload and makes jobs wait less than a fixed weight at the same latency. On one year of measured carbon intensity across eight regions and 143,200 synthetic jobs, CAST cuts emissions by 64.44–71.93% at the knee and by 95.59–95.87% when only carbon is weighed, depending on the forecaster, with no deadline misses and no ceiling violations, also without any reserved headroom. On the Azure VM trace, whose jobs are long and half of whose work cannot be deferred, there are again no violations, and the cost of keeping capacity rises to 16.56 percentage points, more than ten times that on the synthetic workload.

Keywords: Carbon-aware scheduling, Workload shifting, Regional capacity, Linear programming, Rolling horizon, LSTM

1. 서론

데이터센터가 쓰는 전력이 빠르게 늘고 있다. 국제에너지기구는 2024년 약 415TWh였던 전 세계 데이터센터 전력 소비가 2030년 945TWh에 이를 것으로 전망하며 AI를 가장 큰 요인으로 지목했다[1]. 에너지 사용의 투명성이 규제 대상이 된 이상[2] 배출을 줄이는 일은 운영상의 과제가 되었고, 이미 가동 중인 인프라에 신규 설비 없이 적용할 수 있는 수단이 필요하다. 본 논문은 그 수단의 하나인 탄소 인지 스케줄링에서, 탄소 예측이 실측에 가까워질수록 작업이 같은 깨끗한 시각과 리전으로 몰려 용량을 넘기는 문제를 다룬다.

배출량을 정하는 것은 소비 전력량만이 아니다. 전력 1kWh를 만들 때 나오는 탄소량, 곧 탄소집약도는 그 순간 전력망의 발전원 구성에 따라 달라진다. 본 연구의 여덟 리전에서도 2025년 실측 연평균이 인도 543.2 gCO_2/kWh, 프랑스 13.6 gCO_2/kWh로 39.9배 벌어지고, 하루 안에서도 캘리포니아는 날마다 최댓값이 최솟값의 평균 3.93배다. 클라우드는 이미 여러 리전에 흩어져 있고 배치 작업은

완료 시점에 여유가 있으므로, 이 격차를 쓸 조건은 갖추어져 있다.

선행 연구는 이 격차를 시간 축이나 공간 축에서 써 왔다. 시간 축에서는 구글의 CICS[3]가, 공간 축에서는 CASPER[4]와 VMware의 전역 부하 분산[5]이 대표적이다. 두 축을 함께 다룬 Sukprasert 등의 상한 분석[6]은 공간 이동이 지배적이라고 결론지었으나 결합 분석에서는 리전 용량을 두지 않았다. 용량을 다룬 연구는 그 대신 다른 것을 양보한다. CarbonFlex는 단일 클러스터 안의 시간 이동에 한정되고[7], Caspian[8]은 마감을 목적함수의 벌점으로 완화한다. 리전별 상한과 하드 마감을 함께 둔 Zanotto 등[9]은 요청이 도착할 때 그 요청 하나의 자리만 정하고 다시 풀지 않으며, 상한을 어떻게 정할지는 향후 과제로 남겼다.

용량을 빼면 무슨 일이 생기는지는 본 연구의 자료에 그대로 드러난다. Sukprasert 등의 정책을 원문 그대로 다시 구현해 돌리면 작업 143,200건 가운데 142,841건, 곧 99.75%가 프랑스 한 리전으로 몰리고 배출은 96.63% 준다(4.8절). 그러나 이 배치가 위반 없이 돌려면 리전 하나가 동시에 129건을 돌릴 수 있어야 한다. 홈 리전에서 즉시 실행할 때 어느 리전도 13건을 넘긴 적이 없으므로 그 9.9배다. 가장 깨끗한 리전 하나로 전부 보내는 것이 최적해가 되기 때문이며, 계산 위에서만 성립하는 배치다. 용량을 제약으로 두는 순간 문제가 달라진다. 어느 리전이 깨끗한가가 아니라, 그 리전이 받아 줄 수 있는 만큼만 보내고 남은 작업을 어디에 언제 둘 것인가가 된다.

이 문제가 어려운 것은 가장 깨끗한 리전의 가장 깨끗한 시각이 모든 작업에게 같기 때문이다. 리전을 작업이 도착할 때 정하고 실행 시각을 나중에 고르는 2단계 설계에서는, 각 작업이 제 리전의 예측 최저 시각을 따로 고르고 용량은 그 시각이 찼는지를 확인할 때만 쓰인다. 같은 리전의 작업들은 같은 예측을 보므로 같은 골, 곧 하루 중 탄소집약도가 가장 낮은 구간으로 모이고, 예측 곡선의 골이 뾰족할수록 더 많이 모인다. 작업이 각자 저탄소 시각을 고르면 같은 슬롯에 몰린다는 점은 단일 HPC 시설의 시간 이동에서도 지적되었다[10]. 이 설계에서 내부 상한을 물리 상한과 같게 두면, 골을 납작하게 그리는 LSTM에서 18건이던 상한 위반이 전날 실측을 그대로 쓰는 24시간 전 값에서는 125건, 실측값을 그대로 예측으로 쓰는 완전예지에서는 152건이 된다(4.4절). 내부 상한을 0.8배로 줄여 여유를 두어도 완전예지에서 위반은 사라지지 않는다. 위반을 정하는 것은 예측의 오차가 아니라 골의 깊이이며, 더 나은 예측이 더 많은 위반을 부를 수 있다. 용량을 두느냐만이 아니라 어떻게 두느냐가 결과를 가른다.

본 논문은 탄소 인지 스케줄링 시스템 CAST를 제안한다. CAST는 용량을 작업이 시작할 때만 확인하는 대신 이미 정한 배정 전부와 함께 본다. 그러면 같은 골로 몰린 작업은 상한에서 멈추고 나머지는 다음으로 깨끗한 자리로 가므로, 예약 자리를 두지 않아도 상한 위반이 없다(4.4절). 여기에 더해 CAST는 대기 중인 작업 전부를 매 슬롯 한 판에 다시 배정해, 도착 때 한 번 정하고 다시 풀지 않는 설계보다 절감을 8.07%포인트 더한다(4.4절). 이를 위해 CAST는 세 가지를 함께 한다. 용량을 입장 때 거르는 조건이 아니라 대기 중인 모든 작업이 함께 보는 제약, 곧 공유 제약으로 두고, 매 슬롯 아직 시작하지 않은 작업 전부를 다시 풀며, 상한을 넘긴 양을 목적의 첫 항으로 둔다. 뒤의 둘이 요청마다 한 번 자리를 정하고 다시 풀지 않는 선행 연구[9]와 다른 점이다. 그 위에서 탄소와 지연의 가중치는 슬롯마다 고른다.

예측 단계가 여덟 리전의 향후 24시간 탄소집약도를 LSTM으로 내면, 배정 단계가 매 1시간 슬롯마다 아직 시작하지 않은 작업 전부의 실행 리전과 시작 슬롯을 하나의 선형 계획으로 함께 정한다. 상한을 넘긴 양은 목적함수의 첫 항으로 두어 언제나 해가 있게 하고, 한 골에 상한보다 많이 몰리는 배정은 그 항 때문에 애초에 비싸다. 지금 시작하는 작업만 확정하고 나머지는 다음 슬롯에 새 예측으로 다시 푼다. 배치는 예측값으로 결정하되 배출량은 실측값으로 세어 예측 오차의 대가가 결과에 남도록 했다.

본 논문의 기여는 발견 둘과 설계 하나다. 첫째, 탄소 예측이 실측에 가까워질수록 작업이 같은 깨끗한 시각으로 몰려 용량을 넘기며, 위반을 정하는 것은 예측의 오차가 아니라 예측 곡선의 골의 깊이임을 보인다. 용량을 시작할 때만 확인하는 2단계 설계에서 LSTM을 완전예지로 바꾸면 위반 18건이 152건이 되고, 평균 절대 오차로는 24시간 전 값보다 낫지 않은 LSTM이 골을 납작하게 그려 위반을 가장 적게 낸다(4.2절, 4.4절). 미룰 수 있는 작업이 20%에서 60%로 늘면 용량을 무시했을 때의 위반이 세 배가 된다(4.7절). 둘째, 상한이 실제로 걸리는 리전과 시간 이동이 값어치를 만드는 리전이 다름을 보인다. 상한이 걸리는 곳은 리전마다 상한을 달리 배치해도 공간 이동이 모이는 캘리포니아와 프랑스이고, 시간 이동의 절감은 인도와 일본처럼 하루 절대 변동이 큰 리전에서 나며, 둘이 겹치는 리전은 캘리포니아 하나다(4.6절, 4.7절). 선행 정책을 우리가 정한 상한이 아니라 각 정책이 위반 없이 돌기 위해 요구하는 필요 용량으로 다시 재면, CAST의 무릎점보다 많이 줄인 두 정책, CASPER와 Sukprasert 공간+시간은 기준 피크의 2.2배와 9.9배 용량을 전제한다(4.8절). 셋째, 이 둘에 답하는 CAST다. 용량을 시작할 때만 확인하지

않고 이미 정한 배정 전부와 함께 보면 몰림이 상한에서 멈춰 위반이 없다. 그 방법은 용량을 대기 작업 전부가 함께 보는 제약으로 두고 초과를 목적의 첫 항으로 두고 슬롯마다 다시 푸는(rolling) 선형 계획 배정과, 그 위에서 리전과 시작 슬롯을 매 슬롯 함께 정하는 결합형이다(3.3절, 3.4절). 이 가운데 공유 제약은 위반을 없애고, 재풀이는 절감을 더한다. 다시 풀지 않는 대조군도 위반은 없지만 CAST가 합성 워크로드에서 8.07%포인트, Azure 워크로드에서 6.17%포인트 더 줄인다(4.4절, 4.7절). 공유 제약을 쓰면 리전을 도착 때 고정하든 매 슬롯 다시 정하든 시험한 예측기 변형 전부에서 위반이 없고, 예약을 0으로 두어도 같다(4.4절). 그 가운데 결합형은 합성 워크로드에서 같은 고정 α로 리전을 고정하는 쪽보다 비교한 일곱 점 모두에서 0.20~1.54%포인트 근소하게 더 줄이고, 그 차는 상한을 조일수록 벌어져 α=1에서 상한 7일 때 3.04%포인트다(4.5절, 4.7절). 다만 시간 이동이 거의 값어치가 없는 Azure 워크로드에서는 이 차가 무릎점에서 1.13%포인트이고, 트레이스를 다시 뽑은 시드에 따라 −0.95~2.02%포인트로 부호가 바뀌며, α=1에서는 분리형이 1.26%포인트 앞선다(4.7절). 탄소만 보는 α=1에서 완전예지로 한 주를 한 번에 푼 하한과의 격차는 많아야 0.26%포인트다(4.5절). 그 위에서 탄소와 지연의 가중치 α를 고정하지 않고 같은 선형 계획을 열한 번 풀어 슬롯마다 파레토 무릎점에서 고른다(3.4절). 사람이 α를 고르지 않아도 무릎점은 합성 워크로드에서 연간 전선 바로 안쪽에 머물고, Azure 워크로드에서는 같은 지연의 고정 α보다 1.89%포인트 더 줄인다. 어느 쪽이든 같은 지연의 고정 α보다 작업을 덜 기다리게 하며, 이 성질은 세 예측기와 조인 상한, Azure를 합한 여섯 조건 모두에서 같다(LSTM에서 평균 미룸 1.47 대 1.63시간, 4.5절). 상한을 기준 피크의 0.5배에서 3배까지 바꾸어도 무릎점이 고르는 α의 평균은 0.53~0.56에 머문다(4.7절). 무릎점은 예측 곡선의 골이 뾰족할수록 지연을 아끼는 쪽의 점을 고르며, 이 점도 함께 보고한다(4.5절).

1년치 실측 탄소집약도로 시뮬레이션한 결과 CAST는 홈 리전 즉시 실행 대비 배출을 무릎점에서 64.44~71.93%(평균 지연 39.2~47.8 ms), α=1에서 95.59~95.87% 줄였다. 범위는 예측기를 24시간 전 값, LSTM, 완전예지로 바꾼 폭이며, 본 연구의 LSTM으로는 71.93%(47.8 ms)와 95.65%(120.0 ms)다. 어느 경우든 채점 구간의 작업 143,200건 전량이 마감 안에 처리되었고 상한을 넘긴 배정은 없었다.

무릎점의 절감이 α=1보다 낮은 것은 용량 때문에 잃은 몫이 아니라 같은 전선 위에서 지연을 덜 쓰는 쪽을 고른 결과다. 같은 α=1에서 용량을 보지 않으면 절감은 95.97%로 0.31%포인트 더 크지만, 프랑스 한 리전이 동시에 117건, 곧 상한의 6.9배를 돌려야 하고 위반이 46,354건 생긴다. α=1의 95.65%는 상한을 두지 않고 1년치 탄소집약도를 미리 아는 Sukprasert 정책의 96.63%에 0.97%포인트 못 미치지만 기준 피크의 1.25배인 상한 17을 넘지 않는다. 평균 지연이 120.0 ms와 120.4 ms로 거의 같은 CASPER는 95.30%를 내지만 기준 피크의 2.2배인 29건의 동시 실행을 요구한다.

용량 행을 지운 CAST에서는 상한을 넘긴 시간의 93.3%가 공간 이동이 모이는 캘리포니아와 프랑스에서 나고, 캘리포니아에서는 그 74.2%가 현지 9시에서 13시 사이에 몰린다. 총합이 같은 세 가지 리전별 상한 배치에서도 초과 시간의 77~98%가 이 두 리전에서 난다. 시간 이동의 절감은 인도, 일본, 캘리포니아, 독일에 퍼져 있어, 두 목록이 겹치는 리전은 캘리포니아 하나다.

2. 관련 연구

선행 연구는 탄소 인지 스케줄링이 배출을 얼마나 줄일 수 있는지를 여러 방향에서 보여 왔다. 시간 축만 쓰는 연구가 먼저 나왔다. Wiesner 등은 한 데이터센터 안에서 작업을 미루는 것만으로 얻는 절감을 네 지역의 전력 자료로 보였고 자원 제약은 두지 않았다고 밝혔다[11]. CarbonScaler는 한 리전 안에서 작업 하나의 서버 수를 탄소 예보에 맞춰 늘리고 줄이며[12], GAIA는 한 리전 클러스터에서 탄소, 완료 시간, 비용 사이의 맞바꿈을 다룬다[13]. GREEN은 기계학습 클러스터의 스케줄러 안에서 하루 총 용량을 지키며 작업을 저탄소 시각으로 옮긴다[14]. 넷 모두 작업이 어디서 도는지는 바꾸지 않는다. 리전과 시각을 함께 옮기고 리전 용량과 작업 마감을 함께 제약으로 넣고 보면 세 가지가 설계에서 걸린다.

첫째, 두 축을 결합한 선행 연구는 리전 용량과 작업 마감 가운데 하나를 양보한다. Sukprasert 등은 용량이 공간 이동의 이득을 좌우함을 스스로 확인하고도 결합 분석에서는 용량을 두지 않았고[6], Attenni 등도 자원이 항상 가용하다고 가정한다[15]. Lechowicz 등은 작업 하나를 마감 안에서 리전 사이로 옮기는 온라인 문제에 경쟁비 보장을 주었으나, 여러 작업이 자원을 두고 다투는 상황과 데이터센터 용량은 정식화에 없다고 적었다[16]. Hall 등은 하루 전 계획과 실시간 배치로 부하를 시공간으로 옮기며 확률적 성능 보장을 주지만, 대상은 개별 작업이 아니라 집계 부하다[17]. Caspian은 반대로 용량을 하드 제약으로 두는 대신 마감을 목적함수의 벌점으로 완화하며[8], 본 연구 자료에서 재현하면 5,582건, 곧 채점 작업의 3.90%가 마감을 넘긴다(4.8절). 탄력적 확장으로 용량 가정을 방어할 수

있으나, 그 여유는 클라우드 전체가 집계 수준에서 갖는 성질이지 특정 시각의 특정 리전이 갖는 성질이 아니다.

Sukprasert 등은 공간 이득이 작은데도 하루 안의 변동이 커서 순 절감이 의미 있는 예외로 캘리포니아와 버지니아를 지목하였다[6]. 리전 용량이 유한해 공간 이동이 일찍 포화되면 부하가 변동이 큰 리전까지 퍼지므로, 이 예외가 일반적인 상황이 될 수 있다.

둘째, 용량을 두는 연구도 작업이 들어오는 순간에 한 번 판단하고 다시 보지 않는다. Zanotto 등은 본 연구와 가장 가깝다. 리전별 동시 실행 상한, 작업별 하드 마감, 자체 예측기를 갖춘 쿠버네티스 구조로, 요청마다 도착 시점에 기존 배정을 두고 정수 계획으로 리전과 시각을 정하며 그 뒤에는 다시 풀지 않는다. 다만 평가는 1개월 동안 1,000건 묶음을 반복한 것이고 비교 대상은 라운드로빈 하나이며, 상한을 어떻게 정할지는 향후 과제로 남겼다[9]. 이 설정은 4.4절의 대조군으로 재현한다. CICS는 하루 전에 산정한 자원 곡선 하나로 다음 날의 유연한 부하를 묶으므로 개별 작업의 마감을 보지 못한다[3]. CarbonFlex는 단일 클러스터 안에서 용량을 보지 않는 시간 이동이 저탄소 시각에 부하를 모은다는 점을 지적했으나, 리전 사이의 이동은 다루지 않는다[7]. COAST도 작업을 각자 저탄소 시각으로 보내면 같은 슬롯에 몰린다는 데서 출발해, 혼잡을 하드 용량 제약이 아닌 소프트 벌점으로 두고 작업들 사이의 혼잡 게임으로 시작 시각을 나눈다[10]. 대상은 단일 HPC 시설의 시간 이동이며, 예측이 좋아질수록 더 몰린다는 점은 다루지 않는다. 용량을 입장 조건으로만 두면 작업 하나가 들어올 때 그 작업이 고른 시각에 자리가 있는지만 묻고, 여러 작업이 같은 저탄소 시각을 노리고 있다는 사실은 그 시각이 찬 뒤에야 드러난다. 이미 정한 배정을 모두 보는 Zanotto 등의 방식은 이 몰림을 상한에서 멈추게 하지만, 한 번 정한 자리는 나중에 더 깨끗한 자리가 보여도 바꾸지 않는다.

셋째, 탄소와 그 밖의 목적 사이의 가중치가 고정 상수다. CASPER는 탄소와 서버 프로비저닝 비용 사이의 가중치를 고정하며 저자들 스스로 다른 값에서는 결과가 달라질 것이라고 밝혔다[4]. CICS 역시 탄소 비용과 피크 전력 비용의 가중합을 쓰고[3], 두 축을 하나의 정수 계획으로 결합한 Caspian도 가중치를 사람이 고르며 그 값을 신중하게 정해야 한다고 결론지었다[8]. 그 값을 미리 정할 근거는 주어지지 않는다.

본 연구는 이 세 가지를 다음과 같이 다룬다. 마감은 하드 제약으로, 용량은 초과량을 목적함수의 첫 항으로 두어 둘 다 양보하지 않는다(식 (3), (6)). 용량은 입장 조건이 아니라 모든 대기 작업이 함께 보는 제약으로 두어 리전과 시작 시각을 매 슬롯 한 문제로 다시 푼다(3.4절). 가중치는 슬롯마다 파레토 무릎점에서 정한다(3.4절). Zanotto 등과 비교하면 본 연구는 1년 동안 이어지는 작업 흐름, 탄소 인지 선행 정책 넷과의 같은 회계 비교, 상한 값을 바꿔 가며 본 분석을 더 갖추었고, 실제 클러스터 프로토타입은 갖추지 못했다. 본 연구가 재현한 선행 정책 가운데 리전 용량을 입장 조건으로 두는 것은 없다. CICS, CASPER, Sukprasert 등은 정해진 리전 상한 없이 배정하고 Caspian은 용량을 하드 제약으로 두므로, 용량을 시작할 때만 보는 설계에서 생기는 몰림은 이들의 평가에서 드러날 자리가 없었다.

3. CAST 시스템

CAST는 예측기, 로드밸런서, 스케줄러의 세 모듈로 구성된다. 예측기는 리전별로 향후 24시간의 탄소집약도를 산출한다. 로드밸런서는 각 작업을 어느 리전에서 돌릴지를, 스케줄러는 그 리전에서 언제 시작할지를 정한다. 다만 로드밸런서와 스케줄러는 차례로 도는 두 단계가 아니라, 매 1시간 슬롯마다 하나의 선형 계획 안에서 함께 풀리는 두 축이다. 지금 시작하는 작업만 확정해 장부, 곧 리전마다 지금 도는 작업 수를 적어 두는 기록에 올리고, 나머지는 다음 슬롯에 새로 발행된 예측으로 다시 푼다. 그림 1이 세 모듈과 모듈 사이를 오가는 값을 보인다.

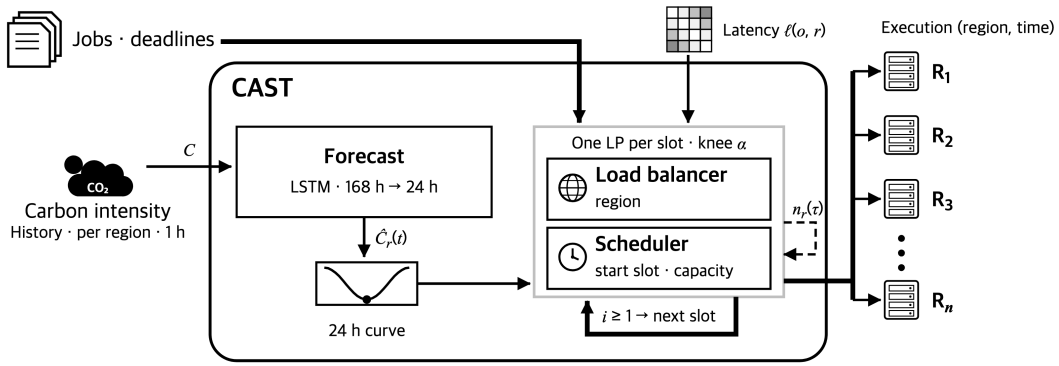


그림 1 CAST의 전체 파이프라인. 굵은 선은 작업, 가는 선은 탄소 자료, 점선은 장부의 리전별 점유가 스케줄러로 다시 들어가는 길이며, 회색 테두리 안의 로드밸런서와 스케줄러는 매 슬롯 하나의 선형 계획으로 함께 풀린다.

Fig. 1 Overall pipeline of CAST. Thick lines carry jobs, thin lines carbon data, and the dashed line returns each region's occupancy from the ledger to the scheduler; the load balancer and scheduler inside the gray frame are solved together as one linear program per slot.

설계 원칙은 셋이다. 첫째, 용량은 입장 때 확인하는 조건이 아니라 대기 중인 모든 작업이 함께 보는 제약으로 두고, 리전과 시각도 그 제약 아래에서 한 문제로 정한다(3.4절). 둘째, 탄소와 지연의 가중치는 고정하지 않는다. 셋째, 배치 결정은 예측값으로, 배출량 산정은 실측값으로 한다. 둘 다 예측값을 쓰면 예측 오차의 대가가 결과에서 사라지기 때문이다.

세 모듈이 지향하는 문제를 먼저 1년치를 한 번에 보는 참조 문제로 적는다. 작업 집합을 J, 리전 집합을 R로 둔다. 슬롯 길이는 탄소집약도 자료의 해상도에 맞춘 1시간이다. 작업 j는 출발 리전 o_j, 제출 시각 s_j, 실행 시간 d_j, 지연 등급 k_j, 지연 예산 L_j를 가지며 마감은 $D_j = s_j + L_j + d_j$다. 리전 r은 동시에 돌릴 수 있는 작업 수의 상한 cap_r을 가진다. 정하는 값은 실행 리전 $r(j)$와 시작 시각 τ_j 둘이고, 리전 r에서 시각 t에 돌고 있는 작업 수를 $n_r(t)$로 쓴다.

목적은 두 층이다. 먼저 상한을 넘긴 양을 최소화하고, 그 안에서 총 배출량과 지연을 줄인다.

$$E = \sum_{j \in J} P \int_{\tau_j}^{\tau_j + d_j} C_{r(j)}(t)\,dt \qquad (1)$$

$$\Lambda = \sum_{j \in J} d_j\, \ell(o_j, r(j)) \qquad (2)$$

$$Z = \sum_{r \in R} \int \max\{0, n_r(t) - \mathrm{cap}_r\}\,dt \qquad (3)$$

여기서 $C_r(t)$는 리전 r의 실측 탄소집약도, P는 작업의 소비 전력, $\ell(o, r)$은 출발 리전에서 실행 리전까지의 왕복 지연이다. E는 총 배출량, Λ는 리전 간 경로를 쓴 시간만큼 지연을 센 값, Z는 상한을 넘긴 작업 수를 시간으로 적분한 초과량이다. Z를 먼저 최소화하고, 같은 Z 안에서 E와 Λ를 함께 줄인다. 두 양은 단위가 달라 정규화와 가중치가 필요한데, 이는 실제로 매 슬롯 푸는 식 (7)~(9)에서 정한다. 제약은 세 가지다.

$$s_j \le \tau_j, \tau_j + d_j \le D_j \forall j \in J \qquad (4)$$

$$\ell(o_j, r(j)) \le L_j \forall j \in J \qquad (5)$$

$$n_r(t) \le \mathrm{cap}_r + z_r(t), z_r(t) \ge 0 \forall r \in R,\ \forall t \qquad (6)$$

식 (4)는 실행이 제출 이후에 시작해 마감 이내에 끝나야 한다는 뜻으로, 마감은 무를 수 없는 하드 제약이다. 식 (5)는 지연 예산을, 식 (6)은 용량을 정한다. 왕복 지연은 응답 경로의 지연으로만 세고 완료 시각에는 더하지 않는다. 상한을 넘긴 양 $z_r(t)$를 변수로 두고 그 적분이 식 (3)이 된다.

식 (6)이 본 연구의 형식화가 선행 연구와 갈라지는 지점이다.

용량을 두지 않으면 최적해는 가장 깨끗한 리전으로 몰린다. 용량을 무를 수 없는 제약으로 두면 마감이 코앞인 작업이 상한보다 많이 몰리는 순간 해가 없다. 초과를 변수로 두면 항상 해가 있고, 용량을 얼마나 어겼는지가 수식 밖의 사정이 아니라 목적함수의 값이 된다.

작업은 둘로 갈린다. 제출 슬롯 말고는 시작할 정수 슬롯이 없는 작업, 곧 $\lfloor D_j - d_j \rfloor \le \lfloor s_j \rfloor$인 작업을 미룰 수 없는 작업이라 부르고, 나머지를 미룰 수 있는 작업이라 부른다. 합성 워크로드에서 미룰 수 없는 작업은 건수로 59.1%지만 실행 시간이 초에서 분 단위라 총 작업시간의 1.3%에 그친다.

식 (1)~(6)을 1년치 전체에 대해 한 번에 풀지 않은 것은 시스템을 온라인으로 두기 위해서다. 한 번에 풀면 모든 작업과 1년치 탄소집약도를 미리 알아야 한다. 그래서 (1)~(6)은 참조 문제로 두고, 실제로는 매 슬롯 3.3절의 식 (9)~(12)를 푼다.

3.1 예측기

예측기는 리전마다 독립인 2층 LSTM이며, 은닉 크기는 64, 드롭아웃은 0.2이고 마지막 은닉 상태를 선형층으로 24개 값에 잇는다. 과거 168시간의 탄소집약도, 무탄소·재생 전력 비율, 시각, 요일, 월의 주기 변수, 공휴일 여부를 받아 향후 24시간을 한 번에 출력한다. 캘리포니아, 텍사스, 독일의 세 리전은 풍속, 일사량, 기온을 입력에 더하고, 향후 24시간의 같은 기상 변수(24×3=72개)도 출력층에 함께 넣는다.

학습은 2021~2023년, 검증은 2024년, 시험은 2025년으로 연도를 끊었고, 스케줄링 실험은 시험 연도 위에서만 돈다. 입력은 학습 연도에만 맞춘 최솟값–최댓값 척도로 0~1에 옮긴다. 손실은 정규화한 값의 평균 제곱 오차이고, Adam(학습률 0.001, 가중치 감쇠 0.0001)으로 배치 64씩 학습하며 기울기 노름을 1.0으로 자른다. 검증 오차가 8 학습 회차(epoch) 동안 줄지 않으면 학습률을 절반으로 낮추고, 15 회차 동안 줄지 않거나 가장 좋았던 회차 뒤 30 회차가 지나면 멈추되, 최소 20 회차, 최대 150 회차를 돈다. 모델은 원래 단위로 되돌린 검증 평균 절대 오차가 가장 작은 회차의 것을 쓰며, 난수 시드는 42다. 구조는 그림 2와 같고, 그림 3은 그 모델이 실제로 낸 예측 한 구간이다.

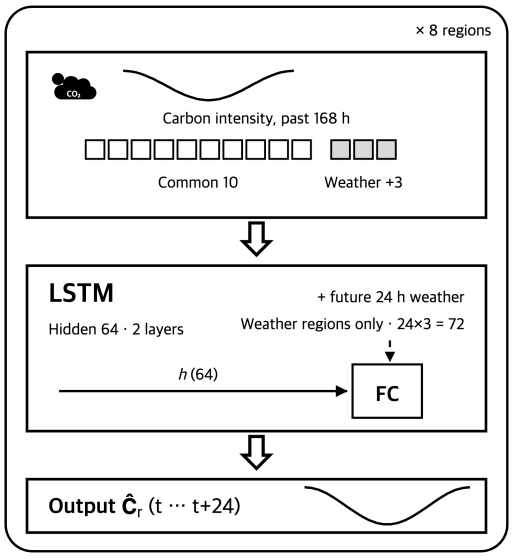


그림 2 리전별 LSTM 구조. 모델은 리전마다 독립이다.
Fig. 2 Per-region LSTM architecture. One model per region.

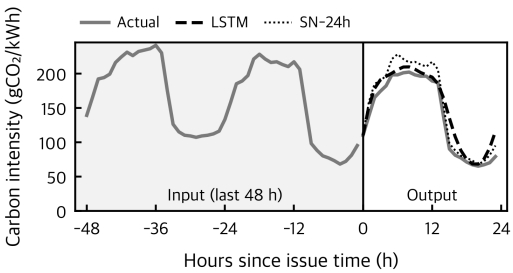


그림 3 리전별 탄소집약도 예측. 캘리포니아의 한 구간이며 왼쪽은 입력 168시간 중 마지막 48시간이다. 이 구간의 평균 절대 오차는 12.2 gCO_2/kWh이고, 점선으로 겹친 24시간 전 값(SN-24h, 어제 같은 시각의 값) 예측은 14.0이다.

Fig. 3 Per-region carbon intensity forecast. One window for California; the left panel shows the last 48 of the 168 input hours; the mean absolute error over this window is 12.2 gCO_2/kWh, against 14.0 for the dashed SN-24h forecast (the value at the same hour on the previous day).

하루 앞 탄소집약도 예측은 선행 연구가 따로 다룬 문제이고[18] 본 연구의 기여가 아니다. 예측기가 뒤의 두 모듈에 넘기는 것은 값 하나하나보다 곡선의 모양, 곧 리전과 시각에 따라 탄소가 어디서 얼마나 깊이 내려가는지다. 로드밸런서와 스케줄러는 그 모양을 보고 자리를 고르기 때문이다. 예측 오차가 스케줄링 결과에

어떻게 이어지는지, 그리고 24시간 전 값을 그대로 쓰는 단순한 예측기와 비교한 결과는 4.2절에 있다.

여기까지가 예측기다. 이제부터는 그 예측을 받아 자리를 고르는 두 모듈을 적는다.

3.2 로드밸런서

로드밸런서는 작업마다 실행 리전을 고른다. 후보는 여덟 리전 전부이며, 식 (5)를 어기는 리전, 곧 출발 리전에서의 왕복 지연이 지연 예산보다 긴 리전은 후보에서 뺀다. 본 연구 자료에서는 지연 예산의 최솟값 1초가 왕복 지연의 최댓값 244 ms보다 커서 이 제약이 한 번도 걸리지 않지만, 예산이 더 짧은 워크로드에서는 이것이 공간 이동의 범위를 정한다. 왕복 지연은 전 구간에서 고정된 8×8 실측 행렬에서 읽는다(4.1절).

후보의 비용은 정규화한 탄소 비용과 지연 비용의 가중합에 실행 시간을 곱한 값이다. 후보는 리전 r과 i의 쌍이며, i는 지금부터 몇 슬롯 뒤에 시작하는지로 3.3절에서 다룬다.

$$c_j(r,i) = d_j\left[\alpha\tilde{C}_j(r,i) + (1-\alpha)\tilde{\ell}(o_j,r)\right] \quad (7)$$

$$\tilde{C}_j(r,i) = \frac{\hat{C}_r^{(t)}(t+i,d_j)}{\max_{r',t'}\hat{C}_{r'}^{(t)}(t')},\ \tilde{\ell}(o,r) = \frac{\ell(o,r)}{244} \quad (8)$$

여기서 $\hat{C}_r^{(t)}(t+i, d_j)$는 슬롯 t에 발행된 예측에서 $t+i$부터 d_j 동안의 평균 탄소집약도이고, 분모는 그 슬롯에 발행된 예측의 리전·지평 전체 최댓값이다. 지연 비용은 전 구간에서 고정된 값인 지연 행렬의 최댓값 244 ms로 나눈다. 탄소 비용만 슬롯별로 정규화하는 것은 슬롯 간 격차의 절대 크기를 가중치 해석에서 떼어내기 위함이다. 고정된 분모를 쓰면 전 리전이 함께 깨끗한 슬롯에서 탄소 항이 일률적으로 작아져 같은 가중치가 슬롯마다 다른 뜻을 갖는다. 실행 시간 d_j를 곱한 것은 긴 작업이 그만큼 오래 배출하고 오래 경로를 쓰기 때문이며, α=1이면 식 (7)의 합이 예측 배출량에 비례한다.

용량을 보지 않는 선행 로드밸런서는 가장 깨끗한 리전에 모든 작업을 보낸다(1장). CAST의 로드밸런서는 각 리전이 받아 줄 수 있는 만큼만 보낸다. 그 '만큼'은 3.3절의 상한 행이 정하고, 남은 작업은 다음으로 깨끗한 리전이나 다른 시각으로 간다. 1년 동안 실제로 만들어진 리전 간 흐름은 4.3절에서 본다.

3.3 스케줄러

스케줄러는 작업마다 시작 슬롯을 고르고, 리전별 장부로 용량을 지킨다. 매 슬롯 t마다 아직 시작하지 않은 작업을 모아 대기 집합 Q를 만든다. 미룰 수 있는 작업은 언제 도착했든 시작하기 전까지 Q에 남고, 이번 슬롯에 도착한 미룰 수 없는 작업도 Q에 들어간다. 슬롯 t에 제출되는 작업은 슬롯 시작 시점에 모두 알려진 것으로 보고 한 번에 배정한다. 최대 1시간을 미리 보는 것이며, 4장의 모든 대조군에 같게 적용한다.

i=0은 지금, 곧 $\max(t, s_j)$에 시작하는 것이고, $i\geq1$은 $t+i$에 시작하는 것이다. i의 상한은 마감과 예측 지평 가운데 이른 쪽, 곧 $h_j = \lfloor\min(D_j - d_j,\ t+H) - t\rfloor$이며 지평 H는 24다. 미룰 수 없는 작업과, 여유가 한 슬롯보다 적게 남은 작업($(D_j - d_j) - t < 1$)은 i=0만 허용한다. 후보를 예측 지평 안으로 제한한 것은 예측이 24시간까지만 있어 그 너머는 비교할 근거가 없기 때문이다.

슬롯 t의 배정 문제는 다음과 같다. x_{jri}는 작업 j를 리전 r에서 i 슬롯 뒤에 시작하게 두는 변수다.

$$\min \sum_{j\in Q}\sum_{r,i} c_j(r,i)x_{jri} + M_A\sum_{r,\tau} z^A_{r\tau} + M_B\sum_{r,\tau\geq t+1} z^B_{r\tau} + \varepsilon\sum_{j\in Q}\sum_{r,i} i\,x_{jri} \quad (9)$$

$$\sum_{r,i} x_{jri} = 1 \quad \forall j \in Q \quad (10)$$

$$\sum_{(j,i)\ni\tau} x_{jri} + n_r(\tau) \leq \mathrm{cap}_r + z^A_{r\tau} \quad (11)$$

$$\sum_{(j,i)\ni\tau,\ j\in Q^{\mathrm{def}}} x_{jri} + n_r(\tau) \leq \mathrm{cap}_r - \rho_r + z^B_{r\tau} \quad (12)$$

변수는 $0 \leq x_{jri} \leq 1$, z^A, $z^B \geq 0$이다. $(j, i) \ni \tau$는 작업 j가 리전 r에서 i 슬롯 뒤에 시작해 d_j 동안 도는 실행이 슬롯 τ와 겹친다는 뜻이고(τ는 슬롯 번호로, 작업의 시작 시각 τ_j와 다르다), Q^{def}는 대기 작업 가운데 미룰 수 있는 작업의 집합이다. $n_r(\tau)$는 이미 시작해 τ에 돌고 있는 작업 수로, 장부에서 읽는다. 식 (10)은 한 작업을 한 자리에만 둔다는 뜻이다. 식 (11)은 식 (6)의 물리 상한으로, 모든 리전 r과 슬롯 $\tau \in [t, t+H]$에 대해 성립한다. 식 (12)는 미룰 수 있는 작업이 미래 슬롯을 끝까지 채우지 못하게 ρ_r 자리를 비워 두는 예약으로, 모든 리전과 $\tau \in [t+1, t+H]$에 대해 성립한다.

초과 변수의 벌점은 $M_A \gg M_B \gg 1$로 두어 두 층 목적을 만든다. 물리 초과를 먼저 없애고, 그다음 예약 침범을, 그 안에서 탄소와 지연을 줄인다. 예약 한 자리를 내주면 그 자리로 여러 작업이 연쇄로 옮겨 갈 수 있으므로, 엄밀한 사전식 순서는 M_B를 대기 작업의 실행 시간 합 $\Sigma_{j\in Q} d_j$ 이상으로 둘 때 보장된다. 실제로는 M_B를 가장 긴 실행 시간의 2배(합성 워크로드에서는 12), M_A를 그 100배로 두었고 이는 실용적 선택이다. 실제 실행에서 두 초과 변수는 1년 8,760 슬롯 어디에서도 양수가 되지 않았다. 마지막 항의 ε은 아주 작은 양수로, 비용이 같은 후보 사이에서 이른 시작을 고르게 할 뿐이다.

용량 행은 두 가지 해상도로 센다. 미래 슬롯($\tau \geq t+1$)은 슬롯을 조금이라도 덮는 작업을 한 자리로 보는 보수적 근사다. 매 슬롯 다시 풀리므로 확정 시점에 바로잡히지만, 그 전까지는 다른 작업의 선택에 영향을

준다. 채점 구간에서 실행 시간이 1시간 미만인 작업 111,142건 가운데 한 슬롯 이상 미뤄져 이런 자리를 차지한 것은 10,529건(9.5%)이다. 지금 슬롯($\tau = t$)만은 이번 슬롯에 작업이 시작하는 순간마다 한 행을 두고, 그 순간에 돌고 있는 작업을 실제 시각 그대로 센다. 그래야 1초짜리 작업 수십 건이 한 시간 자리를 통째로 차지하는 가짜 포화가 생기지 않고, 4.1절의 위반 정의와 같은 자로 잰다.

식 (9)~(12)는 먼저 변수를 0~1 연속으로 두고 선형 계획으로 푼다. 모든 작업이 한 슬롯 안에서 끝난다면 식 (10)(11)은 수송 문제라 제약 행렬이 완전 단모듈이고, 선형 계획의 해가 곧 정수해다[19, 20]. 한 슬롯을 넘는 작업이 있으면 이 성질이 일반적으로 깨지므로, 분수해가 나오면 그 슬롯만 정수 계획으로 다시 푼다. 그런 슬롯의 비율은 4.9절에 있다. 풀고 나면 i=0인 작업만 확정해 리전과 시작 시각을 장부에 올린다. $\geq$1로 배정된 작업은 계획일 뿐이며 다음 슬롯에 다시 Q에 들어간다. 그래서 어떤 미래 슬롯도 미리 예약되지 않지만, 지금 시작하는 긴 작업이 미래 슬롯을 덮는 것은 식 (11)(12)가 $n_i(\tau)$로 센다.

상한 cap_r은 본래 설비 규모에서 와야 하지만 클라우드 사업자는 리전별 설비 규모를 공개하지 않는다. 그래서 상한을 하나의 상수로 유도하지 않고 실험 변수로 둔다. 단위는 모든 작업을 출발 리전에서 즉시 실행했을 때의 리전별 최대 동시 실행 수, 곧 기준 피크이고, 본문은 그 1.25배를 대표 상한으로 쓰며 0.5배에서 3배까지 바꾼 결과를 4.7절에 둔다.

예약 ρ_r도 자료에서 정한다. 미룰 수 없는 작업은 도착한 슬롯에 바로 돌아야 하므로 막을 방법이 없고, 슬롯 τ가 시작될 때 이전 슬롯에 시작해 아직 돌고 있는 작업이 상한까지 차 있으면 τ에 도착하는 작업은 들어갈 자리가 없다. 식 (12)는 미래 슬롯으로 넘어가는 미룰 수 있는 작업을 $\mathrm{cap}_r - \rho_r$ 아래로 묶어 매 슬롯 시작에 ρ_r 자리를 남긴다. 지금 슬롯의 도착은 슬롯 시작 시점에 모두 알므로 $\tau = t$에는 예약을 두지 않는다. ρ_r의 값은 평가 구간 앞의 준비 구간(1월 1~7일)에서 잰 미룰 수 없는 작업의 리전별 최대 동시 실행 수이며, 리전별 최댓값 가운데 가장 큰 값을 전 리전에 같게 쓴다. 합성 워크로드는 3, Azure 워크로드는 13이다. 평가 구간의 자료로 정하지 않았으므로 결과를 보고 고른 값이 아니다. ρ_r을 0에서 6까지 바꾼 결과는 4.7절에 있다.

예약이 있어도 초과는 날 수 있다. 마감이 코앞인 작업이 상한보다 많이 몰리면 식 (11)의 z^{+}가 양수가 된다. 그래서 스케줄러가 하는 일은 용량을 강제하는 것이 아니라 초과를 목적의 첫 항으로 최소화하는 것이며, 생긴 초과는 건수와 시간으로 모두 보고한다(4.4절). 스케줄러는 세 가지를 보장한다. 첫째, 후보 시작 시각이 애초에 $D_j - d_j$ 이하로만 잡히고 여유가 한 슬롯보다 적게 남으면 i=0만 허용되므로, 마감을 넘긴 배정은 탐색 공간에 없다. 둘째, 슬롯이 하나 지날 때마다 남은 여유는 정확히 1씩 줄고 시작 값이 유한하므로 유한한 슬롯 안에 i=0만 남는다. 어떤 작업도 무한히 연기되지 않는다. 셋째, 초과 변수가 있어 식 (9)~(12)는 언제나 해를 가진다.

3.4 로드밸런서와 스케줄러의 결합

식 (9)~(12)는 리전과 시작 슬롯을 한 문제에서 정한다. 둘을 나누어 정하는 설계와 비교하면 이렇게 한 까닭이 드러난다. 작업이 도착할 때 리전을 정하고 시작 시각은 그 리전 안에서 나중에 고르며 용량은 입장 때만 확인하는 2단계 설계에서는, 각 작업이 상한을 보지 않고 제 창 안에서 예측 탄소가 가장 낮은 쪽의 시각을 고른다. 그 시각이 지금인 작업만 빈자리만큼 들어가고 나머지는 다음 슬롯에 다시 고른다. 같은 리전의 작업들은 같은 예측을 보므로 같은 골을 고르고, 예측 곡선의 골이 뾰족할수록 줄이 길어진다. 줄에 선 채 여유가 다한 작업은 자리가 없어도 들어가므로 위반이 생긴다(4.4절). CAST에서는 용량이 입장 조건이 아니라 모든 대기 작업이 함께 보는 제약이어서, 한 골에 상한보다 많이 몰리는 배정은 식 (9)의 초과 항 때문에 애초에 비싸다.

남은 하나는 식 (7)의 가중치 α다. 지연은 예산 안에서도 낮을수록 좋다. 응답이 빠르고, 리전 사이 전송이 줄고, 장애가 퍼지는 범위가 작다. 그래서 탄소와 지연은 서로 다른 두 목적이고, α가 둘의 맞바꿈을 정한다. α=1이면 지연을 보지 않고 탄소만 줄이며, α=0이면 출발 리전에서 즉시 실행하는 것과 같다. CAST는 α를 슬롯마다 파레토 무릎점[21]으로 정한다. 0부터 1까지 0.1 간격의 후보 11개, 곧 $A = \{0, 0.1, \ldots, 1\}$의 각 α에 대해 식 (9)~(12)를 풀어 그 배정의 실행 시간 가중 평균 지연 $\tilde{\lambda}_\alpha = \Sigma_j\, d_j\, \ell(o_j, r(j)) \,/\, \Sigma_j\, d_j$와 예측 배출량 $\tilde{e}_\alpha$를 얻고, 두 축을 후보들 사이에서 각각 0에서 1로 정규화한 뒤 원점에 가장 가까운 후보의 α와 배정을 택한다.

$$\alpha^{*} = \operatorname*{argmin}_{\alpha \in A} \sqrt{\tilde{\lambda}_\alpha^{2} + \tilde{e}_\alpha^{2}} \qquad (13)$$

여기서 $\tilde{\lambda}_\alpha$와 $\tilde{e}_\alpha$는 각각 0~1로 정규화한 평균 지연과 예측 배출이다. 이 규칙은 다중목적 최적화에서 이상점까지의 거리로 타협해를 고르는 방법이다[22]. 물리 초과가 생기는 후보가 있으면 초과가 최소인 후보들 안에서만 고르고, 거리가 같으면 작은 α를 택한다. 홈 리전에 고정된 경우처럼 모든 후보의 지연이 같거나 배출이 같아 한 축이 한 점으로 줄면 α=1로 둔다. 두 축을 모두 실행 시간으로 가중하는 것은 목적함수 식 (1)(2)와

같은 자로 재기 위해서다. 건수로 평균하면 1초짜리 작업 한 건이 6시간짜리 작업 한 건과 같은 표를 갖는다.

리전과 시각을 한 문제로 둔 것이 배정을 어떻게 바꾸는지는 한 슬롯을 열어 보면 분명하다. 그림 4의 슬롯에서 한국을 출발한 작업 하나를 보면, α를 0에서 1까지 옮길 때 배정이 네 구간으로 갈린다(표 1). α가 0.3 이하이면 한국에 남되 6~7시간 뒤로 미뤄 예측 강도를 352에서 329~330 gCO_2/kWh로 낮춘다. 0.4~0.5에서는 지연 30 ms를 들여 일본으로 옮기고 7시간 뒤에 시작해 204로, 0.6~0.9에서는 캘리포니아로 옮겨 2~3시간 뒤에 58~80으로, 1.0에서는 프랑스에서 즉시 시작해 12로 간다. 리전만 바뀌는 것이 아니라 리전마다 고르는 시작 시각도 다르며, α 하나가 로드밸런서와 스케줄러의 선택을 함께 움직인다. 무릎점은 이 슬롯에서 0.5를 골라 지연 30 ms로 예측 강도를 42% 낮춘다.

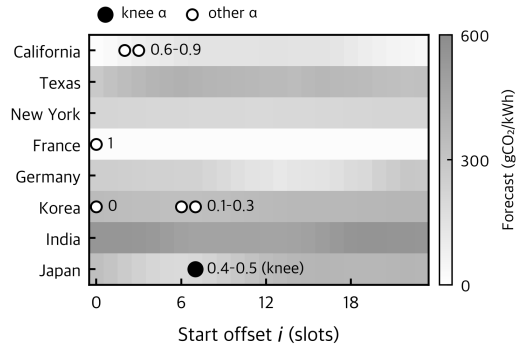


그림 4 한 슬롯에서 α에 따른 배정. 2025년 5월 26일 19시(UTC)에 한국에서 제출된 k=1 작업 하나(실행 17분, 마감까지 14시간)이며, 칸은 리전 × 향후 24 슬롯이고 진할수록 예측 탄소집약도가 높다.

Fig. 4 Assignment by α at one slot. One k=1 job submitted in Korea at 19:00 UTC on 26 May 2025 (17 min run, 14 h to deadline); cells are region × next 24 slots, darker for higher forecast carbon intensity.

표 1 가중치 α에 따른 배정 변화. 그림 4와 같은 슬롯과 작업(출발 리전 한국)이며, 시작은 지금부터 몇 슬롯 뒤인지, 탄소는 예측 강도(gCO_2/kWh), 지연은 ms다.

Table 1 Assignment change by weight α. Same slot and job as Fig. 4 (origin: Korea); start is in slots from now, carbon is forecast intensity in gCO_2/kWh, latency in ms.

α	Region	Start	Carbon	Latency
0.0	Korea	0	352	0
0.1–0.3	Korea	6–7	329–330	0
0.4–0.5	Japan	7	204	30
0.6–0.9	California	2–3	58–80	134
1.0	France	0	12	238

이 작업 하나만 보면 α가 커질수록 탄소가 단조롭게 줄지는 않는다. 같은 슬롯의 다른 작업 29건과 캘리포니아의 자리를 나눠 쓰기 때문이다. α=0.7에서는 캘리포니아의 t+2 자리를 여섯 건이 먼저 채워 이 작업이 t+3으로 밀렸다. 작업마다 따로 골랐다면 이 작업은 예측 강도가 더 낮은 t+2(58)를 골랐을 것이다(t+3은 80).

무릎점은 정규화한 곡선의 기하만 쓰므로 사람이 정한 가중치를 요구하지 않는다. 같은 이유로 무릎점은 곡선의 모양에 따라 움직인다. 예측된 탄소 곡선의 골이 뾰족할수록 탄소 축의 정규화가 달라져, 같은 슬롯에서도 지연 쪽 α를 고른다(4.5절). 슬롯들이 실제로 고른 α의 분포는 4.5절에 있다.

무릎점은 배출이 가장 낮은 지점이 아니라, 지연을 함께 줄이는 쪽의 한 점이다. 무릎점이 고정 α 열한 개의 결과에 대해 어디에 놓이는지는 4.5절에서 본다. Algorithm 1은 세 모듈이 한 슬롯에서 하는 일을 차례로 적은 것이다.

Algorithm 1 Joint rolling assignment

Input: cap_r, ρ_r, H, forecasts $\hat{C}_r^{(t)}(\cdot)$, job stream
Output: (r(j), τ_j) for all j
1: Q ← ∅
2: for t ← 0, 1, 2, … do
3: for each j with ⌊s_j⌋ = t do Q ← Q ∪ {j}
4: for each j ∈ Q do
5: h_j ← ⌊min(D_j − d_j, t + H) − t⌋
6: if j is non-deferrable or (D_j − d_j) − t < 1 then h_j ← 0
7: for each α ∈ {0, 0.1, …, 1} do
8: x^α ← LP solution of (9)–(12)
9: $(\bar{\lambda}_\alpha, \bar{e}_\alpha)$ ← duration-weighted mean latency and forecast emissions of x^α
10: α* ← knee point by (13) with the tie rules of Sec. 3.4; x ← x^{α*}
11: if x is fractional then x ← MILP solution of (9)–(12) at α*

12: for each j ∈ Q with x_{j r 0} = 1 do
13: r(j) ← r; τ_j ← max(t, s_j); record [τ_j, τ_j + d_j) in ledger; Q ← Q \ {j}

비교를 위해 세 가지를 함께 잰다. 첫째는 작업이 도착하는 슬롯에서 로드밸런서가 리전을 정해 고정하고, 그 뒤로는 스케줄러가 시작 슬롯만 같은 문제로 고르게 한 것이다. 비용, 상한, 예약, 예측, 가중치 규칙은 모두 같고 리전을 정하는 시점만 다르며, 4장에서는 분리형이라 부른다. 둘째는 같은 배정에서 식 (11)(12)를 지운 것으로, 저탄소 시각에 몰려도 아무것도 막지 않는다. 고정 α에서의 반사실 상한이며 용량 무시라 부른다. 셋째는 같은 비용, 상한, 무릎점 규칙을 쓰되 선형계획 대신 남은 여유가 적은 작업부터(LLF) 각자 가장 싼 (리전, 시작 슬롯)을 차례로 잡게 한 것으로, 탐욕(greedy) 방식이라 부르고, 이후 탐욕이라 줄여 쓴다. 넷째는 위에서 말한 2단계 설계, 곧 용량을 시작할 때만 보는 가장 단순한 설계로, 예측, 물리 상한, 1시간 선견, 채점은 CAST와 같다. 다른 점은 리전을 작업이 도착할 때 슬롯 단위 정수 계획(가중치는 슬롯마다 무릎점)으로 정해 고정하고, 시작 시각은 그 리전 안에서 탄소와 미룸의 가중합 점수가 가장 낮은 시각(가중치는 지연 등급마다 고정)을 작업마다 따로 고르며, 그 시각이 지금인 작업만 남은 여유가 적은 순으로 내부 상한까지 들여보내고, 여유가 다한 작업은 자리가 없어도 들여보낸다는 것이다. 다시 풀지 않고 예약도 두지 않으며, PuLP와 CBC로 푼다.

3.5 탄소 회계

작업 j의 배출량은 실행 리전의 실측 탄소집약도를 실행 구간에 대해 적분한 값에 소비 전력을 곱한 값이며, 그 합이 식 (1)이다. 실측 계열이 1시간 해상도이므로 적분은 슬롯 단위로 근사한다. 실행이 걸치는 슬롯들의 평균 강도에 실행 시간을 곱한다.

$$E_j \approx P d_j \frac{1}{|T_j|} \sum_{\tau \in T_j} C_{r(j)}(\tau) \qquad (14)$$

여기서 T_j는 실행 구간 $[\tau_j,\ \tau_j + d_j)$이 걸치는 슬롯의 집합이다. 겹침 길이로 가중해 정확히 적분해도 총 배출량은 0.001% 안에서 같다. 소비 전력은 작업당 1kW 상수로 둔다. 상대 비교를 위한 단순화이며, 그 덕에 배출량 차이는 전적으로 실행 리전과 시각에서 온다. 4장의 모든 정책과 대조군은 이 회계 하나로 채점한다.

4. 실험과 분석

4.1 실험 설정

평가는 8개 리전의 2025년 실측 탄소집약도 1년치 위에서 수행한다. 탄소집약도는 Electricity Maps API를 통해 1시간 해상도로 수집하였다[23]. 발전과 설비를 모두 포함하는 전과정 배출계수 체계를 쓰는 공식 방법론이다[24]. 리전은 한국, 캘리포니아, 텍사스, 뉴욕, 프랑스, 독일, 인도, 일본이며, 각 리전에는 Electricity Maps의 다음 구역 자료를 썼다. 캘리포니아 US-CAL-CISO, 텍사스 US-TEX-ERCO, 뉴욕 US-NY-NYIS, 프랑스 FR, 독일 DE, 한국 KR, 일본 JP, 인도 IN이다. 미국 동부 리전은 탄소 자료로 뉴욕 전력망(NYISO) 구역을, 지연 자료로 Azure East US 행을 쓰며, 이후 뉴욕이라 부른다. 2025년 자료는 1월 8일부터 받을 수 있어, 1월 1~7일은 1월 8일의 하루 모양을 되풀이해 채운 준비 구간이고 그 기간에는 예측과 실측이 같다. 이 2025년 자료는 LSTM 학습에 쓴 2021~2024년 자료와 다른 경로로 받았다. 그래서 시뮬레이션은 1월 1일부터 돌리되 모든 지표는 1월 8일 0시부터 연말까지 8,592시간에 대해 세고, 그 전에 제출된 작업은 채점에서 뺀다.

네트워크 지연은 8×8 리전 간 왕복 지연 실측 행렬(최댓값 244 ms, 비대각 최솟값 12 ms)을 Azure 통계[25]에서 가져왔다. 전 구간에서 고정이며, 부하에 따라 지연이 바뀌는 효과는 모델에 없다.

워크로드는 둘이다. 합성 워크로드는 총 146,000건이며 리전당 하루 50건이 제출되고, 채점 구간에 들어오는 것은 143,200건이다. 등급별 비율(k=1부터 3:1:1:2:3)은 공개 트레이스에 지연 등급 표시가 거의 없어(Azure 트레이스는 0.5%) 정한 합성 파라미터다. 지연 예산은 등급 안에서 로그균등으로, 실행 시간은 로그정규 분포에 상·하한을 두어 뽑는다(표 2). 도착에는 하루 주기가 있다. k=3~5는 현지 13시, k=1~2는 현지 2시를 봉우리로 하는 표준편차 3시간의 정규분포에서 70%가 오고 나머지 30%는 하루 전체에 고르게 온다. 미룰 수 없는 작업이 건수로 59.1%지만 작업시간으로는 1.3%라, 시간 이동의 여지는 작업시간 기준으로 거의 전부에 있다. 공간 이동은 식 (5)가 걸리지 않아 모든 작업에 열려 있다.

표 2 지연 등급별 예산과 실행 시간. 예산은 등급 안에서 로그균등, 실행 시간은 로그정규에 상·하한을 두어 뽑는다. 중앙값과 상·하한은 초 단위이고 건수는 1년 전체다.

Table 2 Latency budget and job duration by tier. Budgets are log-uniform within each tier; durations are log-normal and clamped (seconds). Counts cover the full year.

k	Budget	Median d	σ	Clamp	Jobs
1	12–24 h	5,400	0.9	300–21,600	43,830
2	2–8 h	1,800	0.9	60–14,400	14,458
3	2–10 min	180	0.8	5–1,800	14,716
4	15–60 s	30	0.7	1–300	29,244
5	1–5 s	5	0.6	1–60	43,752

두 번째 워크로드는 Azure의 2019년 VM 트레이스[26]에서 만들었다. 29일씩 13장을 이어 붙여 1년을 만들되 장마다 다른 VM 11,600개를 뽑았고(추출률 0.49%), 마지막 장은 연말에서 잘려 146,016건이다. 실행 시간은 VM이 생성되어 삭제될 때까지의 간격 그대로이고(300초 단위), 도착 시각도 트레이스의 하루 모양을 그대로 따른다. 기준 배출은 213,513.4 kg으로 합성의 7.5배이고, 이 워크로드의 %포인트는 이 값에 대한 차다. 트레이스에 없는 두 값은 모형으로 채웠다. 출발 리전은 여덟 리전에 고르게 무작위로 주고, 지연 등급은 트레이스가 지연 민감 여부를 표시한 0.5%만 그 표시를 따르며 나머지는 합성 워크로드와 같은 비율로 준다. 도착과 길이는 실측이고 지연 예산은 모형이다.

두 워크로드는 실행 시간에서 서로 다른 끝을 덮는다. 실행 시간의 중앙값은 합성 81초, Azure 1,500초이고, 95 백분위수는 합성 13,074초, Azure 74,400초(20.7시간)다. 합성은 VM 트레이스가 300초 해상도라 보지 못하는 짧은 작업을, Azure는 하루를 넘는 긴 작업을 담는다. 총 작업시간은 Azure가 합성의 7.8배이고, 미룰 수 없는 작업이 차지하는 작업시간 비중은 합성 1.3%, Azure 51%로 정반대다.

등급 비율 3:1:1:2:3은 두 원칙을 따랐다. 첫째, 시간 이동이 가장 잘 듣는 등급 1과 지연에 가장 민감한 등급 5를 같은 30%로 두었다. 다만 지연 예산이 모두 왕복 지연보다 길어 공간 이동은 모든 작업에 열려 있으므로, 이 워크로드는 공간 축에 유리하다(4.3절의 공간만 61.57% 대 4.4절의 시간만 18.78%). 네트워크 지연에 상한을 두어 공간 이동을 막았을 때의 결과는 4.7절에 있다. 둘째, 1초에서 하루까지 다섯 자릿수에 걸친 지연 예산의 어느 구간도 비지 않도록 가운데 세 등급을 1:1:2로 얇게 두었다. 그 결과 합성 워크로드는 건수로는 짧은 작업이, 작업시간으로는 긴 작업이 대부분인 꼴을 갖는다. 15초 미만 작업은 건수의 32.2%지만 작업시간의 0.1%이고, 1시간 이상 작업은 건수의 22.4%로 작업시간의 85.2%를 차지한다. Azure 트레이스에서도 10분 미만 작업이 건수의 20.0%, 작업시간의 0.3%이고 하루 이상 작업이 건수의 4.3%로 작업시간의 68.1%를 차지해 같은 꼴이며, Google 트레이스에서도 대다수 작업이 몇 분 안에 끝나지만 전체 기간 내내 도는 작업이 2,000개를 넘고 전체의 7%인 생산 우선순위 작업이 나머지 전부보다 많은 자원을 쓴다[27]. 미룰 수 있는 작업의 비율을 20~60%로 바꾼 결과는 4.7절에 있다.

상한은 3.3절대로 실험 변수다. 기준 피크는 합성 워크로드에서 캘리포니아 11, 텍사스 10, 뉴욕 10, 프랑스 10, 독일 11, 한국 12, 인도 13, 일본 12이고, Azure 워크로드에서는 가장 큰 것이 34다. 대표 상한은 가장 큰 기준 피크의 1.25배를 올림한 값으로 합성 17, Azure 43이며, 모든 리전에 같게 주므로 리전마다 자기 피크의 1.3~1.7배가 된다. 어떤 정책이든 채점은 이 물리 상한으로 한다. CAST가 안에서 쓰는 예약 ρ는 채점 기준이 아니다.

지표는 다음과 같다. 총 배출량은 식 (1)을 3.5절의 회계로 잰 값이고, 절감률은 기준 대비 백분율이다. 용량 위반은 상한을 넘긴 배정 건수다. 작업 j가 시작하는 순간 같은 리전에서 이미 돌고 있는 작업이 상한 이상이면 j를 위반으로 센다. 같은 시각에 시작한 작업은 서로를 보고, 같은 시각에 끝나는 작업은 먼저 뺀다.

초과 시간은 리전별로 동시 실행이 상한을 넘은 시간의 합이다. 필요 용량은 그 정책의 배정이 만든 리전별 최대 동시 실행 수의 최댓값으로, 그 정책이 위반 없이 돌려면 필요한 상한이다. 채점 상한과 무관한 값이라 선행 정책 비교의 자로 쓰며, 1초짜리 순간 피크의 영향을 가리기 위해 시간 가중 99.9 백분위수를 함께 적는다. 이 밖에 마감 위반 건수, 공간 이동이 더한 왕복 지연의 평균과 95 백분위수, 제출에서 시작까지 미룬 시간을 잰다.

몰림, 곧 여러 작업이 같은 깨끗한 시각을 함께 노리는 정도는 세 값으로 설명한다. 리전-일마다 실측 탄소가 가장 낮은 1시간의 최대 동시 실행을 평균한 값, 그 시각에 동시 실행이 상한에 닿은 리전-일의 비율, 미룬 작업 가운데 그날 탄소가 가장 낮은 세 시각에 시작한 작업의

비율이다. 이 셋은 몰림의 모양을 보일 뿐이며, 몰림이 해로운지는 절감과 위반으로 가린다.

무릎점과 고정 α의 비교에는 같은 지연 짝을 쓴다. 무릎점의 평균 지연과 0.5 ms 안에서 같은 지연을 내는 고정 α를 이분법으로 찾아 1년 전체를 돌린 실행이다. 두 실행의 차는 무릎점이 더 배출한 양으로 적고, 7일 묶음을 되뽑아 2,000번 다시 계산한 블록 부트스트랩의 95% 신뢰구간이 0보다 위에 있으면 무릎점이 뒤진다고, 0을 포함하면 차이가 없다고, 0보다 아래에 있으면 이긴다고 판정한다. 이 신뢰구간은 한 실행 안의 주 단위 변동을 잰 것이며, 워크로드를 다시 뽑는 변동은 시드를 바꾼 다섯 벌의 실행으로 따로 잰다(4.7절).

실험은 Python 3.14.7과 SciPy 1.18.0의 HiGHS 풀이기로 Apple M1 Max(성능 코어 8, 효율 코어 2, 메모리 32 GB, macOS 15.7.7)에서 돌렸다. 4.4절의 2단계 설계는 PuLP 3.3.2와 CBC 2.10.3으로 같은 기계에서 돌렸다. 모든 표의 숫자는 작업별 기록에서 별도 스크립트로 다시 계산해 대조했다.

본 연구의 코드, 합성 워크로드 생성기, Azure 트레이스 변환 스크립트, 실행 설정은 저장소 https://github.com/HyeonJeong-S/carbon-aware-scheduler 에 공개한다. 탄소집약도 자료는 Electricity Maps 이용 약관에 따라 재배포하지 않고 수집 스크립트와 전처리 코드만 제공한다.

4.2 예측기 평가

예측기는 값을 얼마나 맞히는가와 그 예측이 배정을 얼마나 잘 이끄는가로 나누어 본다. 2025년 시험 연도 전 지평의 평균 절대 오차는 여덟 리전 평균 26.11 gCO_2/kWh로, 직전값을 그대로 쓰는 예측(48.47)과 168시간 전 값을 쓰는 예측(35.68)보다 낮고 24시간 전 값을 쓰는 예측(SN-24h, 25.27)과는 비슷하다. 리전별로는 한국, 텍사스, 뉴욕, 독일 네 곳에서 앞서고 캘리포니아, 프랑스, 인도, 일본 네 곳에서 뒤진다. 그림 3의 구간 오차 12.2는 캘리포니아 연간 오차 22.69의 한 단면이다. 값의 오차만 보면 LSTM은 24시간 전 값보다 낮다고 할 수 없다.

배정에 미치는 효과는 다르다. 예측기를 24시간 전 값, LSTM, 완전예지로 바꾸어 CAST를 돌린 결과가 표 3에 있다. α를 0.5로 고정하면 절감이 61.82%, 62.99%, 63.46%로 예측이 좋을수록 높다. 평균 절대 오차가 LSTM보다 조금 낮은 24시간 전 값이 절감은 1.17%포인트 낮으므로, 배정 결과를 정하는 것은 값의 오차만이 아니다. 곡선의 모양을 재면 24시간 창 안의 순위상관은 LSTM 0.629, 24시간 전 값 0.677이고, 예측 최저 시각이 실측 최저 시각과 맞는 비율은 21.5%와 30.3%로 오히려 24시간 전 값이 높다. 둘이 크게 갈리는 것은 골의 깊이다. 예측한 하루 최대−최소를 실측의 것으로 나눈 비가 24시간 전 값은 1.001, LSTM은 0.647로, LSTM은 골을 실측의 65% 깊이로 납작하게 그린다. 탄소만 보는 α=1에서는 95.59%, 95.65%, 95.87%로 차이가 0.28%포인트 안이다. 무릎점에서는 64.44%, 71.93%, 67.20%(평균 지연 39.2, 47.8, 40.7 ms)로 순서가 뒤집히는데, 이는 예측의 질이 아니라 무릎점 규칙이 곡선의 모양에 따라 다른 α를 고르기 때문이며 4.5절에서 가른다. 어느 예측기에서든 CAST의 위반은 0이다.

표 3 예측기와 설계에 따른 절감률(%)과 평균 지연(ms). 괄호는 상한 17을 넘긴 배정 건수이고 —는 돌리지 않은 칸이다. 2단계의 η는 내부 상한을 물리 상한의 몇 배로 두었는지이며, η=1.0이면 내부 상한이 물리 상한과 같고, η=0.83은 내부 상한 14로 CAST의 미래 슬롯 상한 17−ρ와 같다. Arrival-fixed는 작업이 도착한 슬롯에 이미 정한 배정을 모두 보고 리전과 시각을 한 번 정한 뒤 다시 풀지 않는 Zanotto 등[9] 식 대조군이다.

Table 3 Savings (%) and mean latency (ms) by forecaster and design. Parentheses give assignments past the ceiling of 17; a dash marks a run not performed. For the two-stage design, η is the internal ceiling as a fraction of the physical one; at η=1.0 the two coincide, and η=0.83 gives an internal ceiling of 14, equal to CAST's future-slot ceiling 17−ρ. Arrival-fixed is a control in the manner of Zanotto et al. [9]: each job's region and start are fixed once in its arrival slot against all existing assignments and never re-solved.

Design	SN-24h	LSTM	Oracle
Two-stage, η=0.8	63.04 (0) / 36.2	63.78 (6) / 36.2	64.38 (19) / 36.2
Two-stage, η=0.83	—	64.31 (6) / 36.2	64.92 (31) / 36.2
Two-stage, η=1.0	64.54 (125) / 36.2	65.26 (18) / 36.2	66.02 (152) / 36.2
Arrival-fixed, knee	—	63.85 (0) / 41.4	63.72 (0) / 36.6
Arrival-fixed, α=1	—	91.36 (0) / 110.5	—
Decoupled, knee	43.77 (0) / 19.5	48.88 (0) / 23.4	46.62 (0) / 19.9
Decoupled, α=1	95.57 (0) / 119.9	95.34 (0) / 120.0	95.85 (0) / 119.9
CAST, fixed α=0.5	61.82 (0) / 32.7	62.99 (0) / 33.9	63.46 (0) / 32.6
CAST, knee	64.44 (0) / 39.2	71.93 (0) / 47.8	67.20 (0) / 40.7
CAST, α=1	95.59 (0) / 120.0	95.65 (0) / 120.0	95.87 (0) / 119.9
Capacity ignored, knee	—	73.39 (24,993) / 47.8	69.25 (33,964) / 41.5
Capacity ignored, α=1	—	95.97 (46,354) / 120.4	—

4.3 로드밸런서 평가

본 연구의 일곱 방식을 같은 회계와 같은 상한 17로 비교하였다. 기준은 홈 리전 즉시 실행, 시간만은 홈을 고정하고 스케줄러만 쓴 것, 공간만은 시각을 고정하고 로드밸런서만 쓴 것이며, 분리형, CAST, 용량 무시, 탐욕은 3.4절에서 정의했다. 이 일곱 이름을 이후 모든 표와 그림에서 그대로 쓴다. 각 방식은 무릎점과 α=1 두 가중치로 돌렸고 결과는 표 4에 있다. 이 절은 로드밸런서의 몫을, 4.4절은 스케줄러의 몫을, 4.5절은 둘을 합친 몫을 다룬다.

표 4 일곱 방식의 절감률, 평균 지연, 최대 동시 실행. 각 칸은 무릎점 / α=1이며, 최대 동시 실행은 리전별 최댓값 가운데 가장 큰 값으로 4.8절의 필요 용량과 같은 자다. 상한은 17이며, 시간만은 지연 축이 없고 탐욕은 무릎점에서만 돌려 한 값만 적었다.

Table 4 Savings, mean latency and peak concurrency of the seven methods. Each cell is knee / α=1; Peak is the largest per-region maximum concurrency, the same measure as the required capacity in Section 4.8. The ceiling is 17; Temporal-only has no latency axis and Greedy was run at the knee only, so each has a single value.

Method	Savings (%)	Latency (ms)	Peak
Baseline	0 / 0	0 / 0	13 / 13
Temporal-only	18.78 / —	0 / —	17 / —
Spatial-only	61.57 / 93.96	40.4 / 120.1	17 / 17
Decoupled	48.88 / 95.34	23.4 / 120.0	17 / 17
CAST	71.93 / 95.65	47.8 / 120.0	17 / 17
Capacity ignored	73.39 / 95.97	47.8 / 120.4	63 / 117
Greedy	71.08 / —	47.6 / —	17 / —

로드밸런서만으로도 절감의 대부분이 나온다. 공간만은 무릎점에서 61.57%(40.4 ms), α=1에서 93.96%(120.1 ms)를 줄이며, 두 경우 모두 최대 동시 실행이 상한 17을 넘지 않는다. 용량을 보지 않는 공간 이동인 CASPER는 지연이 120.4 ms로 같은 α=1 쪽에서 95.30%로 1.34%포인트 더 줄이지만, 그 배치는 리전 하나가 동시에 29건을 돌려야 성립한다(4.8절). 다만 그 1.34%포인트에는 용량을 지킨 대가와 함께 목적함수와 구현의 차이도 섞여 있다. 제약 두 줄만 다른 짝은 아래의 용량 무시다. 공간 이동이 실제로 만든 흐름은 그림 5에 있다. CAST는 채점 구간 작업의 58.4%인 83,665건을 출발 리전 밖에서 돌렸고, 리전별로 실행된 몫은 프랑스 51.6%, 캘리포니아 27.3%로 두 리전이 78.9%를 차지한다. 가장 큰 흐름은 독일에서 프랑스(옮긴 물량의 21.3%)와 인도에서 프랑스(20.2%)이며, 텍사스와 일본, 한국의 작업은 주로 캘리포니아로 간다.

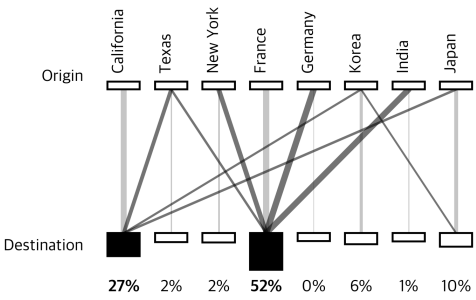


그림 5 로드밸런서가 만든 리전 간 흐름. CAST(무릎점)의 채점 구간 143,200건에서 옮긴 물량의 93.9%를 차지하는 흐름 8개이며, 회색 세로선은 홈에 남은 몫, 아래 상자 높이는 리전별로 실행된 몫에 비례하고 검은 칠은 그 몫이 20%를 넘는 리전이다.

Fig. 5 Inter-region flow created by the load balancer. The 8 flows carrying 93.9% of moved jobs among the 143,200 scored jobs under CAST (knee α). Gray verticals are the share kept at the origin, lower box heights are proportional to each region's share of executions, and black fill marks regions above 20%.

CAST에서도 용량을 지키는 비용은 작다. 용량 무시는 무릎점에서 73.39%, α=1에서 95.97%로 CAST보다 1.47%포인트와 0.31%포인트 더 줄이지만, 최대 동시 실행이 63건과 117건으로 상한의 3.7배와 6.9배이고 상한을 넘긴 배정이 24,993건과 46,354건이다. 넘친 자리는 거의 모두 로드밸런서가 작업을 모으는 리전에서 난다(4.6절). 작업당 1kW 가정에서 CAST는 1년에 무릎점에서 20,589.1 kg, α=1에서 27,381.0 kg을 덜 내보낸다. 이 회계는 리전 사이로 옮긴 작업의 전송 에너지를 넣지 않았다. 옮긴 작업마다 데이터 1 GB를 보내는 데 드는 0.06 kWh(2015년 기준 추정값으로, 해마다 줄고 있어 보수적이다)[28]를 목적지 리전의 탄소집약도로 더해도, CAST의 절감은 무릎점에서 71.93%가 71.16%로, α=1에서 95.65%가 95.20%로 1%포인트 안에서 줄고, 비교한 아홉 실행(표 4에서 기준과 탐욕을 뺀 실행)의 순서는 바뀌지 않는다.

4.4 스케줄러 평가

스케줄러만 쓴 시간만은 홈 리전을 떠나지 않고 시각만 옮겨 18.78%를 줄인다. 평균 2.17시간 미루며, 최대 동시 실행은 상한 17에 닿되 넘지 않는다. 스케줄러가 용량을 어떻게 다루는지는 캘리포니아의 한 날을 열어 보면 분명하다. 그림 6은 동시 실행이 연중 가장 높았던 날인 7월 6일의 탄소 곡선, 그날 시작해 미뤄진 작업 가운데 창이 가장 넓은 9건, CAST의 시각별 동시

실행을 위아래로 겹친 것이다. 그날 CAST의 동시 실행은 하루 24시간 가운데 8시간 동안 상한 17에 닿아 있고, 그 시간은 현지 9시에서 16시, 곧 태양광이 만드는 한낮의 골이다.

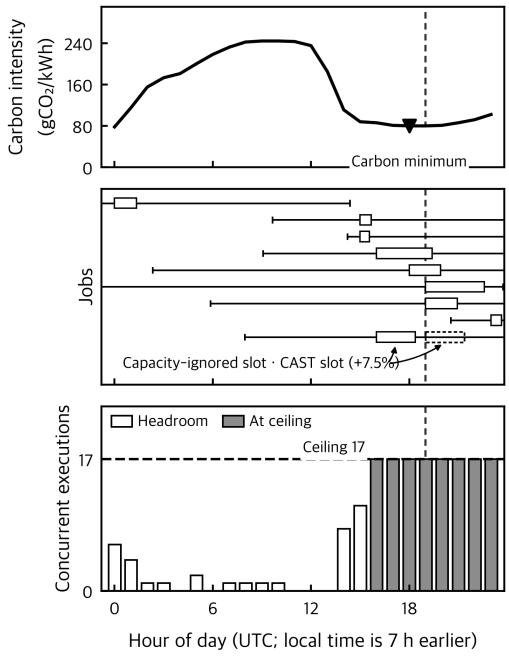


그림 6 용량이 저탄소 시각으로의 몰림을 막는 방식. 캘리포니아의 한 날(7월 6일)이며 그날 시작해 미뤄진 작업 중 창이 19.5시간 이상인 9건이다. 가는 선이 창, 굵은 막대가 CAST의 실행 구간, 점선 상자가 용량 무시에서 받은 시각이며 굵은 테두리가 본문의 예다. 아래 단은 CAST의 동시 실행과 상한 17이다.

Fig. 6 How capacity keeps jobs from piling into the low-carbon hours. One day in California (6 July), showing the 9 deferred jobs started that day with windows of at least 19.5 h. Thin lines are windows, thick bars CAST's executed intervals and dashed boxes the slots received under capacity ignored, with the bold outline marking the example in the text; the bottom panel shows CAST's concurrency against the ceiling of 17.

작업들이 가고 싶어 하는 시각은 모두 같은 골이며, 용량을 보지 않았다면 받았을 시각(점선 상자)과 CAST가 실제로 준 시각(굵은 막대)이 어긋나는 것은 그 골이 이미 상한까지 찼기 때문이다. 굵은 테두리로 표시한 작업은 현지 0시 58분에 제출된 2.36시간짜리 작업이다. 용량을 보지 않았다면 그날 탄소가 가장 낮은 현지 12시(80 gCO_2/kWh)에 시작했겠지만, 골이 이미 차 있어 CAST는 이 작업을 골의 앞자락인 현지 9시(86 gCO_2/kWh)로 당겼다. 이 작업 하나만 보면 배출이 7.5% 늘지만 상한은 넘지 않는다.

용량을 입장 때 확인하는 2단계 설계는 다르게 반응한다. 3.4절의 2단계 설계에서 내부 상한을 물리 상한 17과 같게 두면, 예측기를 LSTM에서 완전예지로 바꿀 때 절감은 65.26%에서 66.02%로 조금 오르지만 위반은 18건에서 152건으로, 초과 시간은 1.4시간에서 9.2시간으로 는다(표 3). 완전예지는 같은 시각을 최선으로 보는 작업을 늘려 그 시각 앞의 줄을 길게 만든다. 내부 상한을 0.8배인 13으로 줄여 여유를 두어도 위반은 6건에서 19건으로 늘 뿐 사라지지 않는다. CAST의 미래 슬롯 상한(17−3)과 같은 14를 주어도 LSTM에서 6건, 완전예지에서 31건이다.

작업이 도착한 슬롯에 이미 정한 배정을 모두 보고 리전과 시각을 한 번 정한 뒤 다시 풀지 않는 Zanotto 등[9]의 설정에서는 완전예지에서도 위반이 0이다(표 3). 위반을 없애는 데는 이미 정한 배정을 함께 보는 제약 하나로 충분하며, 2단계의 위반은 작업이 고른 시각에 자리가 있는지를 시작할 때만 확인하는 데서 온다. 매 슬롯 다시 푸는 것의 몫은 절감에 있다. CAST는 이 대조군보다 무릎점에서 LSTM 8.07%포인트, 완전예지 3.48%포인트, α=1에서 4.29%포인트를 더 줄이고, 평균 미룸도 2.21시간에서 1.47시간으로 짧다.

위반을 정하는 것은 예측의 오차가 아니라 골의 깊이다. 평균 절대 오차가 LSTM과 비슷한 24시간 전 값은 전날의 실측 곡선을 그대로 쓰므로 골이 뾰족하고, 같은 2단계 설계에서 위반이 125건이다. LSTM을 하루에 한 번만 발행하면 60건, 완전예지에 LSTM만 한 잡음을 더하면 28건이다. LSTM이 18건으로 가장 적은 것은 골을 실측의 65% 깊이로 납작하게 그리고(4.2절), 예측 최저 시각이 발행이 바뀔 때마다 절반 가까이(48.4%) 옮겨 가기 때문이다. 24시간 전 값과 완전예지는 15.4%만 옮겨, 같은 시각을 최선으로 보는 작업이 그만큼 더 몰린다. 내부 상한을 0.8배로 줄이면 위반은 예측기에 따라 0~19건으로 눌리지만 완전예지의 19건은 남는다. 오차로는 24시간 전 값보다 낫지 않은 LSTM이 몰림을 가장 덜 만든 것은 예측기의 값어치가 오차가 아니라 골의 모양에 있음을 뜻한다.

리전을 먼저 정하는 분리형도 몰림 자체는 같은 방향으로 움직인다. 미룬 작업이 그날 탄소 하위 세 시각에 시작하는 몫이 LSTM 23.9%, 24시간 전 값 34.5%, 완전예지 52.5%로 늘고, 완전예지와 LSTM을 섞는 비율을 바꾸면 그 사이를 오간다. 그러나 분리형은 용량을 공유 제약으로 두므로 여덟 예측기 변형 전부에서 위반이 0이다.

용량을 모든 대기 작업이 함께 보는 제약으로 두면 이 증가가 없다. 리전을 도착 때 고정하는 분리형과 CAST 모두, 무릎점과 α=1 각각에서 예측기를 세 가지로 바꾼 열두 실행이 전부 위반 0이다. 예약을 0으로 두어도 두 설계와 세 예측기의 여섯 조합 모두 위반이 0이고, 절감은 예약 3일 때와 0.3%포인트 안에서 같다. Azure 워크로드에서도 예약 없이 위반이 0이다(4.7절). 위반이 없는 것은 예약이 아니라 용량을 함께 보는 제약 때문이다. 탄소만 보는 α=1에서는 두 설계 모두 완전예지가 가장 높은 절감을 내고, 세 예측기의 차이도 0.6%포인트 안이다. 작업 대부분이 하루 변동이 작은 프랑스로 가서(홈에 남는 작업 12.5%) 같은 시각으로 모일 일 자체가 적기 때문이다. 몰림이 문제가 되는 것은 작업이 집 근처의 변동이 큰 리전에 남는 무릎점 쪽이다.

용량을 보지 않을 때와 CAST의 차이를 한 주로 늘려 보면 그림 7과 같다. 7월 6일이 든 주(day 184~190)에서 용량 무시는 한낮마다 상한을 넘겨 최대 63건에 이르고, CAST는 같은 시각을 17건에서 멈춘다.

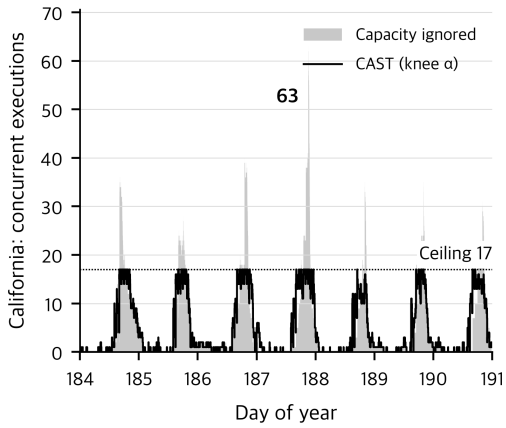


그림 7 캘리포니아의 동시 실행, 용량 무시와 CAST. 연중 한 주(day 184~190)이며 회색 채움이 용량 무시, 검은 선이 CAST, 점선이 상한 17이다.

Fig. 7 California concurrent executions under capacity ignored and CAST. One week of the year (days 184–190); gray fill is capacity ignored, the black line CAST, and the dashed line the ceiling of 17.

분리형에서 용량 행을 지우면 위반이 13,845건, 초과 시간이 615시간 생기고 최대 동시 실행이 45건에 이르지만, 절감은 48.42%로 용량이 있을 때의 48.88%보다 오히려 조금 낮다. 상한이 없으면 무릎점 규칙의 후보 열한 개가 모두 비교 대상에 들어가 정규화가 바뀌고, 무릎점이 α 평균 0.438에서 0.434로, 평균 지연이 23.4 ms에서 22.9 ms로 지연 쪽으로 옮겨 간다. 골에 몰린 작업이 더 깨끗한 시각을 얻는 것도 아니다. 분리형에서 용량 행은 절감을 깎지 않고 위반만 없앤다.

작업 143,200건은 모든 방식에서 전량 마감 이내에 처리되었다. 후보 시각을 마감 이내에서만 찾으므로 설계상의 결과다. CAST의 무릎점에서 미룬 시간은 평균 1.47시간이고 작업의 41.6%가 홈 리전에서 돌며, 평균 지연은 47.8 ms다. α=1에서는 홈에 남는 작업이 12.5%로 줄고 평균 지연이 120.0 ms로 는다.

4.5 결합과 무릎점

로드밸런서와 스케줄러를 한 문제로 합친 값은 가중치에 따라 다르다. 같은 규칙으로 고른 무릎점끼리 비교하면 CAST는 공간만보다 10.35%포인트를 더 줄이고, α=1에서는 1.70%포인트를 더 줄인다(표 4). 지연을 아끼는 쪽일수록 시간 축의 값어치가 커지는 것이다. α=1에서는 작업 대부분이 가장 깨끗한 리전으로 가므로 시각을 옮겨 더 얻을 것이 적다. 다만 두 무릎점은 고른 α의 평균이 공간만 0.488, CAST 0.556으로 달라서 같은 α의 비교가 아니다. 분리형의 무릎점이 공간만보다 낮은 것(48.88% 대 61.57%)도 시간 축을 더해서가 아니라 무릎점이 고른 자리 때문이다. 분리형의 무릎점은 α 평균 0.438로 지연 쪽을 골라 작업의 62.2%를 홈에 남기며, 같은 분리형에 고정 α=0.5를 주면 62.02%(33.48 ms)로 공간만을 넘는다.

설계끼리는 같은 α에서 비교해야 한다. 상한이 기준 피크의 1.25배인 이 조건에서 두 설계의 전선은 거의 겹친다. 그러나 같은 α끼리 비교하면 CAST가 α=0.4부터 1까지 일곱 점 모두에서 0.20~1.54%포인트 덜 배출하고, 신뢰구간은 모두 0을 포함하지 않는다(표 5). 다만 이 차는 워크로드 시드를 바꿀 때의 폭(표준편차 0.28~0.43%포인트, 4.7절)과 같은 크기라, 결합형은 근소하게 더 줄인다고 보는 것이 맞다. 이 차는 합성 워크로드의 결과이며, 시간 이동이 거의 값어치가 없는 Azure 워크로드에서는 성립하지 않는다(4.7절). 차이는 α=0.6에서 가장 크고 α=0.8 이상에서는 0.2~0.3%포인트로 작아진다. 리전과 시각을 함께 정하는 값어치는 이 조건에서 작지만 0이 아니며, 상한을 조이거나 작업이 길어질 때의 차이는 4.7절에서 본다.

표 5 같은 고정 α에서 CAST와 분리형의 비교. 칸은 절감률(%)이며 두 실행의 평균 지연 차는 α=0.6의 1.00 ms를 빼면 0.54 ms 안이다. 차는 분리형 배출에서 CAST 배출을 뺀 값(양수 = CAST가 덜 배출, kg)이며 괄호는 7일 단위 블록 부트스트랩의 95% 신뢰구간, pp는 기준 배출 대비 %포인트다.

Table 5 CAST versus the decoupled design at the same fixed α. Cells are savings (%); the two runs differ in mean latency by at most 0.54 ms except 1.00 ms at α=0.6. Diff. is decoupled minus CAST emissions in kg (positive = CAST emits less) with 95% 7-day block-bootstrap intervals; pp is the gap in percentage points of baseline emissions.

α	CAST	Decoupled	Diff. (95% CI)	pp
0.4	38.64	37.80	238 (66–434)	0.83
0.5	62.99	62.02	276 (205–351)	0.96
0.6	77.69	76.15	441 (355–528)	1.54
0.7	88.99	88.29	200 (114–282)	0.70
0.8	93.51	93.31	59 (10–109)	0.20
0.9	95.24	94.98	73 (35–115)	0.25
1.0	95.65	95.34	91 (52–135)	0.32

선형 계획이 얼마를 더하는지는 탐욕과 비교하면 드러난다. 탐욕은 무릎점에서 71.08%를 내며 위반이 없고 최대 동시 실행도 17이다. 평균 지연이 0.29 ms 안으로 같은 CAST는 탐욕보다 242 kg, 기준 배출의 0.84%포인트를 덜 배출한다(95% 신뢰구간 160~322 kg). 탐욕도 완전예지에서는 65.28%(40.2 ms)로 내려가 CAST와 같은 방향으로 움직인다. 무릎점끼리 쌓아 보면 공간 이동만으로 61.57%, 시간 축을 한 문제로 합친 탐욕이 9.51%포인트를 더하고, 선형 계획이 탐욕보다 0.84%포인트를 더한다. 구현을 맞추지 않은 비교에서는 선형 계획 쪽이 더 빨랐다(4.9절).

무릎점이 고정 α보다 나은지는 한 축으로 물을 수 없다. α=1이 탄소의 최솟값이고 α=0이 지연의 최솟값이기 때문이다. 고정 α 열한 개를 각각 1년 동안 돌려 얻은 (평균 지연, 절감)의 점이 그림 8에 있다. 열한 점을 이은 선은 보간일 뿐이다. α=0.45가 내는 점이 0.4와 0.5를 잇는 선보다 아래에 떨어질 수도 있으므로, 그 선은 그림의 보조선으로만 두고 판정에는 쓰지 않는다. 판정은 같은 지연 짝으로 한다(4.1절).

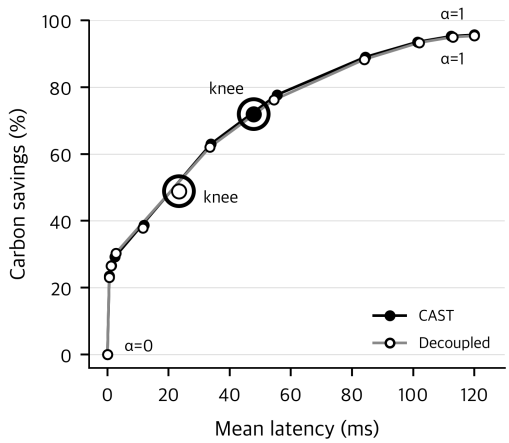


그림 8 고정 α 전선과 슬롯별 무릎점. ●와 검은 선은 CAST, ○와 회색 선은 분리형의 고정 α 열한 점(0, 0.1, …, 1)이며, 선은 점을 이은 보조선이고 과녁 표식이 각 설계의 무릎점이다.

Fig. 8 Fixed-α frontiers and per-slot knee points. Filled dots with the black line are CAST and hollow dots with the gray line the decoupled design at eleven fixed α (0, 0.1, …, 1); lines only connect the points, and target markers are each design's knee point.

분리형에서는 무릎점이 같은 지연의 고정 α에 뒤진다. 무릎점의 평균 지연은 작업당 23.4 ms, 실행 시간 가중 20.45 ms다. 작업당 지연으로 맞춘 짝인 고정 α=0.4375는 22.95 ms에서 55.49%를 내어, 무릎점(48.88%)이 이 짝보다 1,894 kg, 기준 배출의 6.62%포인트를 더 배출한다(95% 신뢰구간 1,543~2,256 kg 더). 무릎점 규칙이 쓰는 실행 시간 가중 지연으로 맞춘 짝(α=0.4156, 20.57 ms)에서도 무릎점이 666 kg(2.33%포인트) 더 배출해(188~1,141 kg 더) 결론은 같다. 분리형의 무릎점은

α를 너무 낮게 골라 작업을 홈에 남긴다. 분리형 무릎점이 예측기에 따라 CAST와 다른 방향으로 움직이는 까닭(표 3)은 가르지 않았다.

CAST에서는 차이가 훨씬 작다. 무릎점의 평균 지연은 작업당 47.85 ms, 실행 시간 가중 47.48 ms다. 작업당 지연으로 맞춘 짝인 고정 α=0.5687은 47.71 ms에서 72.86%를 내며, 무릎점은 이 짝보다 268 kg(0.93%포인트) 더 배출하지만 신뢰구간이 0을 포함해(252 kg 덜 ~ 808 kg 더) 차이가 없다. 실행 시간 가중 지연으로 맞춘 짝(α=0.575, 가중 지연 47.08 ms, 73.79%)에서는 534 kg(1.87%포인트) 더 배출해(31~1,072 kg 더) 근소하게 뒤진다. 0.5687과 0.575는 1년을 돌려 찾은 값이지만, 아래에서 보듯 첫 주만 보고 골라도 같은 자리에 온다. 무릎점은 그 대신 작업을 덜 기다리게 한다. 평균 미룸이 1.47시간으로 짝의 1.63시간보다 짧고, 95 백분위수도 9.98시간 대 10.96시간이다. 같은 규칙이 분리형에서는 6.62%포인트 뒤지는 것과 견주면, LSTM 예측 아래에서 슬롯별 무릎점은 결합 설계에서 같은 지연의 고정 α에 버금가는 자리를 고른다.

결합 설계에서 이 판정이 예측기에 따라 달라지는지 보려고, 같은 지연 짝을 세 예측기 모두에서 찾았다(표 6). LSTM에서는 앞서 본 대로 차이가 없다. 그러나 24시간 전 값과 완전예지에서는 같은 지연의 고정 α가 무릎점보다 더 줄인다. 24시간 전 값에서는 무릎점이 짝(α=0.5375)보다 1,006 kg, 기준 배출의 3.51%포인트를 더 배출하고(95% 신뢰구간 480~1,597 kg 더), 완전예지에서는 짝(α=0.5438)보다 931 kg(3.25%포인트)을 더 배출한다(395~1,449 kg 더). 따라서 무릎점이 같은 지연의 고정 α와 구별되지 않는다는 결과는 대표 상한 17의 LSTM 예측 아래에서만 선다. 반면 작업을 덜 기다리게 하는 것은 세 예측기 모두에서 같다. 평균 미룸은 무릎점과 짝이 24시간 전 값에서 2.45 대 2.62시간, LSTM에서 1.47 대 1.63시간, 완전예지에서 2.13 대 2.31시간이다. 상한을 조여도 같은 모양이다. LSTM에서 상한을 10과 7로 줄이면 상한 17에서 구별되지 않던 차이가 신뢰구간 0 위로 올라와 무릎점이 짝보다 2.64%포인트와 2.19%포인트 더 배출하지만, 평균 미룸은 1.80 대 2.05시간, 2.20 대 2.33시간으로 여전히 무릎점이 짧다. Azure 워크로드에서는 방향이 반대다. 무릎점은 같은 지연의 짝(α=0.5438)보다 4,033 kg, Azure 기준 배출의 1.89%포인트를 덜 배출하고(95% 신뢰구간 820~6,939 kg 덜), 평균 미룸도 1.59 대 1.73시간으로 짧다. 지연이 조금 더 큰 이웃 α=0.55도 45.43%로 무릎점보다 낮다. 무릎점이 고정 α보다 나은지도 워크로드가 정한다. 무릎점이 고르는 자리가 예측 곡선의 모양에 따라 움직인다는 아래의 결과와 같은 방향이다.

표 6 예측기와 상한에 따른 무릎점과 같은 지연 짝(CAST). 위 세 행은 합성 워크로드의 상한 17, 다음 두 행은 LSTM에서 상한을 10과 7로 줄인 것, 마지막 행은 Azure 워크로드(상한 43, 괄호는 Azure 기준 배출 대비)다. Azure 짝의 지연차는 0.52 ms이며, 지연이 더 큰 이웃 α=0.55(44.67 ms, 45.43%)와 견주어도 결론이 같다. 짝은 작업당 평균 지연이 무릎점과 0.5 ms 안에서 같은 고정 α이고, 미룸은 무릎점 / 짝의 평균 미룸(시간), Δ는 무릎점이 짝보다 더 배출한 양(괄호는 기준 배출 대비 %포인트), 신뢰구간은 7일 블록 부트스트랩 2,000회의 95% 구간이다.

Table 6 Knee point versus its same-latency pair by forecaster and ceiling (CAST). The top three rows use the ceiling of 17; the next two use LSTM with the ceiling cut to 10 and 7, and the last row is the Azure workload (ceiling 43; percentage points of the Azure baseline). The Azure pair differs by 0.52 ms in latency, and the conclusion is the same against the higher-latency neighbour α=0.55 (44.67 ms, 45.43%). The pair is the fixed α whose mean per-job latency is within 0.5 ms of the knee's; Delay is the mean delay of knee / pair (h), and Δ is the knee's extra emission over the pair (percentage points of the baseline in parentheses), and the CI is the 95% interval of 2,000 seven-day block bootstrap draws.

Condition	Knee % (ms)	Pair α (ms)	Pair %	Delay (h)	Δ kg (%p)	95% CI (kg)
SN-24h	64.44 (39.18)	0.5375 (39.65)	67.95	2.45 / 2.62	1,006 (3.51)	480 to 1,597
LSTM	71.93 (47.85)	0.5687 (47.71)	72.86	1.47 / 1.63	268 (0.93)	−252 to 808
Oracle	67.20 (40.70)	0.5438 (40.74)	70.46	2.13 / 2.31	931 (3.25)	395 to 1,449
LSTM, cap 10	66.11 (44.60)	0.5540 (44.54)	68.75	1.80 / 2.05	756 (2.64)	336 to 1,191
LSTM, cap 7	56.79 (38.46)	0.5375 (37.97)	58.98	2.20 / 2.33	627 (2.19)	367 to 861
Azure	46.14 (43.56)	0.5438 (43.04)	44.25	1.59 / 1.73	−4,033 (−1.89)	−6,939 to −820

고정 α 하나를 예측기와 무관하게 쓰면 어떻게 되는지도 보았다. LSTM에서 찾은 짝 α=0.5687을 다른 두 예측기에 그대로 주면 24시간 전 값에서 72.58%(46.4 ms), 완전예지에서 73.97%(46.0 ms)를 낸다. 고정 α는 예측기를 바꿔도 지연 46~48 ms, 절감 72.6~74.0%에서 안정하다. 무릎점은 골이 뾰족한 이 두 예측기에서 지연을 아끼는 쪽으로 물러선 자리(39.2 ms와 40.7 ms)를 골라, 이 고정 α보다 절감이 8.15%포인트와 6.77%포인트 낮은 대신 지연은 7.2 ms와 5.3 ms, 평균 미룸은 0.17시간과 0.21시간 짧다. 두 축 모두에서 무릎점을

앞서는 고정 α는 없었으므로 이 고정 α가 무릎점을 지배하지는 않으며, 무릎점은 지연 쪽의 한 점을 고른다.

평가 구간의 첫 주(1월 8~14일)만 보고 무릎점과 같은 지연이 나오도록 고른 고정 α=0.5625도 연간 71.92%(46.27 ms)로 무릎점과 같은 자리에 온다. 같은 지연의 고정 α는 1년을 다 보아야만 찾을 수 있는 값이 아니다. 합성 워크로드에서 무릎점의 절감이 같은 지연의 고정 α보다 0.93~3.51%포인트 낮은 것은, 슬롯마다 고른 무릎점을 모은 자리가 연간 전선에서 지연 쪽으로 조금 안쪽에 놓이기 때문으로 읽힌다. 슬롯마다의 전선을 합한 것이 연간 전선은 아니다.

〖표 6 의 나머지 조건 행: 시드 1~4〗

같은 무릎점 규칙을 1년이 끝난 뒤 고정 α 열한 점에 한 번 적용하면, 두 설계 모두 α=0.5를 고른다. 실행 시간으로 가중한 지연 축에서도 같다. 1년 전체를 미리 알고 하나의 α를 골랐다면 0.5였다는 뜻이다. CAST의 슬롯별 무릎점은 이 사후 무릎점(33.86 ms, 62.99%)보다 지연을 14.0 ms 더 쓰고 8.94%포인트 더 줄이는 쪽에, 분리형의 슬롯별 무릎점은 사후 무릎점(33.48 ms, 62.02%)보다 지연을 덜 쓰는 쪽에 놓인다. 어느 쪽도 사후 무릎점을 두 축 모두에서 이기거나 지지 않는다.

α는 슬롯마다 하나를 골라 그 슬롯의 모든 리전과 작업에 같이 쓴다. 슬롯이 고른 α는 평균 0.556이고, 채점 구간 8,592 슬롯 가운데 0.6이 48.3%, 0.5가 40.7%, 0.7이 6.3%로 세 값이 95.3%를 차지한다. 0.4~0.7 밖을 고른 슬롯은 54개(0.63%)이고 0, 0.1, 0.9, 1은 한 번도 고르지 않았다.

이 분포는 예측 곡선의 모양에 따라 움직인다. 완전예지에서는 α=0.6을 고른 슬롯이 4,150개에서 2,986개로 줄고 0.5를 고른 슬롯이 3,496개에서 4,449개로 늘며, 아시아의 낮인 UTC 3~10시에 평균 α가 0.53에서 0.47~0.51로 가장 많이 내려간다. 완전예지는 골이 뾰족해 탄소 축의 정규화가 달라지고, 무릎점이 지연 쪽으로 반칸 물러난다. LSTM 실행에서도 α는 하루 안에서 움직인다. UTC 13~17시에는 평균 0.59~0.61로 오르고 UTC 2~8시에는 0.52~0.53으로 내려가, 캘리포니아의 태양광 골이 열리는 시간대에 탄소 쪽으로 기운다.

4.2절에서 무릎점의 순서가 뒤집힌 까닭이 이것이다. 완전예지 무릎점(67.20%)이 LSTM 무릎점(71.93%)보다 4.72%포인트 낮은 차이를 가르기 위해, 한 실행이 슬롯마다 고른 α를 다른 예측기의 실행에 그대로 주었다. LSTM 예측에 완전예지의 α 경로를 주면 69.23%(43.8 ms), 완전예지 예측에 LSTM의 α 경로를 주면 71.33%(45.2 ms)다. 같은 α 경로에서 예측기만 바꾸면 절감은 0.59~2.03%포인트 달라지고, 같은 예측기에서 α 경로만 바꾸면 2.69~4.13%포인트 달라진다. 두 순서를 평균하면 차이의 약 72%가 α 경로에서, 약 28%가 예측에서 온다. 상한을 없앤 실행에서도 완전예지 무릎점은 73.39%에서 69.25%로 내려가므로, 이 차이는 용량이 아니라 무릎점이 고르는 자리에서 온다.

예측을 LSTM에서 완전예지 쪽으로 조금씩 섞어 가면 이 움직임이 연속으로 보인다. 완전예지의 몫을 25%, 50%, 75%로 늘리면 무릎점 절감이 71.71%, 70.76%, 68.93%로, 평균 지연이 46.9, 45.3, 42.9 ms로 차례로 내려가 완전예지의 67.20%, 40.7 ms에 이른다. 위반은 모두 0이다.

몰림 자체는 공유 제약에서도 남는다. 완전예지 예측에 LSTM의 α 경로를 준 실행은 탄소 최저 시각의 동시 실행이 평균 6.67건으로 LSTM 실행의 4.68건보다 많고, 그 시각에 상한에 닿은 리전-일도 23.92%로 11.31%의 두 배다. 그런데 절감은 71.33%로 LSTM과 0.59%포인트밖에 차이 나지 않고 위반은 0이다. 몰린 작업은 골을 상한까지 채우는 데서 멈추고 나머지는 다음으로 깨끗한 자리로 간다. 공유 제약은 몰림을 지우지 않고 해롭지 않게 만든다.

온라인 구조가 얼마를 내주는지는 한 주를 한 번에 푼 하한과 비교해 잰다. 캘리포니아의 피크가 든 주(day 184~190)에 제출된 작업 2,800건을 같은 상한 17로 두고, 탄소만 보는 α=1과 완전예지 예측 아래에서 각 작업의 시작 후보를 마감까지 전부 열어 한 문제로 풀었다. 이때 실행 구간이 슬롯을 통째로 덮을 때만 그 슬롯의 자리로 세어 용량을 느슨하게 두었으므로, 이 문제의 해는 어떤 배정도 넘을 수 없는 하한이다. 하한은 그 주 기준 배출 558.1 kg의 98.37%를 줄이고, 같은 작업과 조건에서 CAST는 98.12%를 줄여 격차는 0.26%포인트(1.44 kg)다. 하한이 느슨하고 그 주 이전 작업의 점유도 넣지 않았으므로, 24시간 지평과 슬롯마다 다시 푸는 온라인 구조의 대가는 이 주에서 많아야 0.26%포인트다. 탄소와 지연을 함께 보는 고정 α=0.5에서도 같은 주를 한 번에 풀어 견주었다. 이때는 목적에 지연이 들어가고 탄소 항의 정규화도 한 주 전체의 최댓값으로 해서 롤링과 목적함수가 같지 않으므로, 하한이라 부르지 않고 두 축을 따로 적는다. 한 번에 푼 해는 73.14%(45.11 ms), CAST는 72.52%(45.78 ms)로, CAST가 절감에서 0.62%포인트, 지연에서 0.67 ms 뒤진다.

4.6 상한이 걸리는 자리

상한이 어느 리전에서 걸리는지는 용량 무시 실행에서 상한을 넘긴 시간을 리전별로 나누어 보면 드러난다. 상한이 모든 리전에 같으면 작업이 모이는 리전에서 넘치는 것은 거의 정해진 일이고, 실제로 상한을 넘긴 시간의 55.6%가 캘리포니아, 37.7%가 프랑스, 6.7%가 일본에서 나며 나머지 다섯 리전은 한 번도 넘기지

않는다. 이 두 리전은 연평균이 149.2와 13.6 gCO_2/kWh로 가장 깨끗하고, CAST에서 실행된 몫이 가장 큰 두 리전(78.9%)이기도 하다. 리전마다 상한을 달리 두어도 이 순서가 유지되는지는 4.7절에서 본다.

이 절의 발견은 그 자리가 시간 이동이 값어치를 만드는 자리와 다르다는 데 있다(그림 9(a)). 홈을 고정하고 시각만 옮긴 시간만의 절감 5,377 kg을 출발 리전별로 나누면 인도 21.5%, 일본 21.2%, 캘리포니아 19.4%, 독일 18.5%, 텍사스 12.3%다. 이 다섯은 하루 안의 절대 변동, 곧 날마다 최댓값과 최솟값의 차를 평균한 값이 148.1~197.4 gCO_2/kWh로 가장 큰 다섯 리전이고, 시간 이동 절감의 92.9%를 낸다. 작업당 1kW로 절대 배출량을 세는 회계에서는 예상되는 비례다. 프랑스는 날마다 (최댓값−최솟값)/일평균을 1년 평균한 상대 변동이 0.97로 캘리포니아(1.06) 다음으로 크지만, 연평균이 13.6 gCO_2/kWh로 낮아 절대 변동이 11.8로 가장 작고 시간 이동 절감의 0.3%만 낸다.

두 목록에 함께 드는 리전은 캘리포니아 하나다. 캘리포니아는 CAST에서 실행된 몫의 27.3%를 받는 목적지이면서, 자기 작업의 시각을 옮겨 시간 이동 절감의 19.4%를 내는 출발지이고, 용량을 두지 않으면 초과 시간의 55.6%가 나는 리전이다. 시간 이동의 이득이 큰 리전 대부분에서는 상한이 걸리지 않고, 상한이 걸리는 곳은 공간 이동이 모이는 곳이다.

두 리전은 넘치는 시각도 다르다(그림 9(*b*)). 캘리포니아는 상한을 넘긴 시간이 모두 일광절약시간을 반영한 현지 8시에서 17시 사이에 있고, 그 74.2%가 9시에서 13시까지의 다섯 시간에 몰린다. 태양광이 만드는 한낮의 골로 작업이 함께 모인 결과다. 프랑스는 반대로 현지 밤에 넘친다. 가장 큰 시간대가 2시로 14.3%이고, 0시에서 4시까지가 57.4%다. 프랑스의 하루 최저는 수요가 낮은 밤에 오고, 독일·인도·뉴욕에서 온 작업이 그 시각으로 모인다. 그래서 캘리포니아에서만 한낮의 골을 두고 공간 이동으로 온 작업과 시간 이동으로 미룬 작업이 같은 자리를 다툰다.

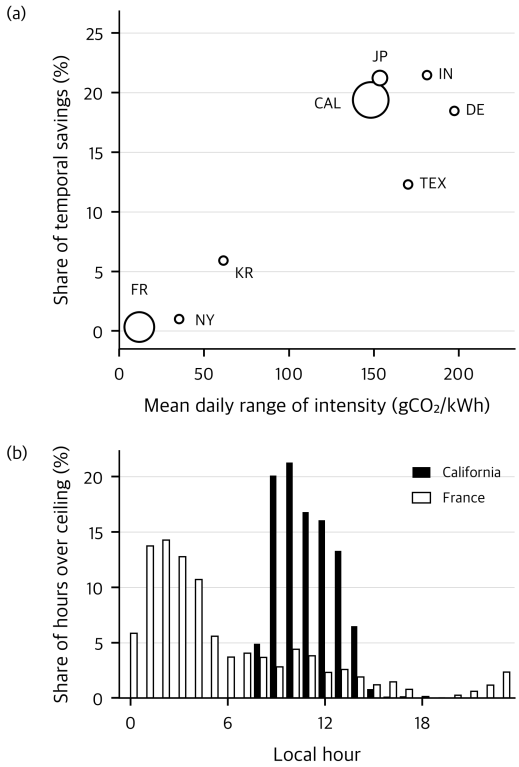


그림 9 상한이 걸리는 자리와 시간 이동이 값어치를 만드는 자리. (a) 여덟 리전의 하루 절대 변동(가로, 채점 구간의 일별 최대−최소 평균)과 시간 이동 절감 몫(세로, 시간만의 출발 리전별 감소량)이며, 원 넓이는 용량 무시(무릎점)에서 상한 17을 넘긴 시간의 몫이고 작은 ○는 넘긴 적이 없는 리전이다. (*b*) 같은 실행에서 상한을 넘긴 시각의 현지 시간(일광절약시간 반영) 분포로, 리전별 합을 100%로 두었고 채운 막대가 캘리포니아, 빈 막대가 프랑스다.

Fig. 9 Where capacity binds and where temporal shifting pays. (a) Daily absolute swing (x; mean daily max−min over the scored window) against share of temporal-shift savings (y; temporal-only reduction by origin) for the eight regions; circle area is the share of hours over the ceiling of 17 under capacity ignored (knee α), and small circles mark regions that never exceed it. (b) Local-hour distribution of those hours (daylight saving applied), normalised per region; California filled, France hollow.

4.7 결론이 성립하는 범위

상한을 바꾸면 결과가 상한값에 얼마나 매였는지 드러난다. 상한을 모든 리전에 같게 기준 피크 13의 0.5배(7)부터 3배(39)까지 일곱 단계로 바꾸어 CAST를

돌렸고, 결과는 그림 10(a)에 있다. 예약 ρ는 미룰 수 없는 작업이 준비 구간에서 필요로 한 자리이므로 상한과 무관하게 모든 단계에서 3으로 고정했다. 일곱 단계 모두 위반과 초과 시간이 0이다. 기준 피크의 절반인 상한 7에서는 홈 리전 즉시 실행조차 713번 상한을 넘기지만, CAST는 무릎점에서 56.79%를 줄이며 한 번도 넘기지 않는다.

절감은 상한을 늘릴수록 오르다가 기준 피크의 2배 근처에서 멈춘다. 무릎점 절감은 상한 7, 10, 13, 17, 20, 26, 39에서 56.79%, 66.11%, 70.16%, 71.93%, 72.72%, 73.21%, 73.42%이고, 3배에서는 73.42%로 용량 무시(73.39%)를 0.03%포인트 넘는데, 무릎점이 상한에 따라 α를 다시 고르기 때문이며 용량 무시는 고정 α에서만 상한이다. 용량을 지키는 비용, 곧 용량 무시와의 차는 대표 상한 17에서 1.47%포인트, 기준 피크와 같은 13에서 3.24%포인트, 절반인 7에서 16.61%포인트이고, 2배인 26에서는 0.19%포인트로 거의 사라진다. α=1에서도 상한 7, 10, 13에서 76.30%, 89.71%, 94.18%, 20에서 95.80%, 39에서 95.96%로 같은 꼴이다.

상한이 조여도 무릎점이 고르는 α는 거의 움직이지 않는다. 슬롯이 고른 α의 평균은 상한 7에서 0.530, 17에서 0.556, 39에서 0.554이고, 평균 지연도 38.5 ms에서 48.0 ms 사이다. 상한이 조이면 무릎점이 지연 쪽으로 물러서는 대신 갈 자리가 없는 작업이 홈에 남아, 홈 리전에서 도는 작업이 41%에서 50%로 는다. 리전을 도착 때 고정하는 분리형은 상한을 풀어도 무릎점 절감이 상한 7~26에서 48.44~49.71%에 머문다. 분리형의 무릎점은 α를 0.44 안팎으로 두어 작업을 홈에 남기므로, 상한이 결과를 정하지 않는다.

예약이 없어도 대표 상한 17에서는 위반이 나지 않는다. 예약 ρ를 0, 2, 3, 4, 6으로 바꾸면 절감은 72.06%, 72.16%, 71.93%, 72.00%, 71.53%로 ρ=3과 0.40%포인트 안에서 오르내리고, 평균 지연은 47.7~48.0 ms이며, 위반은 모두 0이다. 매 슬롯 i=0만 확정하므로 아직 확정하지 않은 미래 슬롯의 계획은 언제든 비켜 줄 수 있고, 이미 시작해 돌고 있는 작업만으로 슬롯이 찬 때가 없었기 때문이다. 예약이 바꾸는 것은 미룰 수 있는 작업 가운데 마감까지 남은 여유가 한 슬롯 아래로 떨어져 그 자리에서 시작해야 했던 작업의 수로, ρ를 0에서 6으로 늘리면 4,805건에서 3,931건으로 준다. 상한을 조여도 마찬가지다. 상한 7과 10에서 예약을 0으로 두면 절감은 58.30%와 66.85%로 예약 3일 때보다 1.51%포인트와 0.74%포인트 높고, 위반은 여전히 0이다. 상한 13에서도 70.10%로 예약 3일 때와 거의 같다. 대신 마감 직전에 시작이 강제되는 작업이 상한 7에서 13,479건으로, 예약 3일 때(4,625건)의 세 배 가까이 된다. 어느 상한에서도 위반이 없는 것은 예약에 기대지 않으며, 예약은 절감을 조금 내주고 마감 직전으로 밀리는 작업을 줄이는 안전장치다.

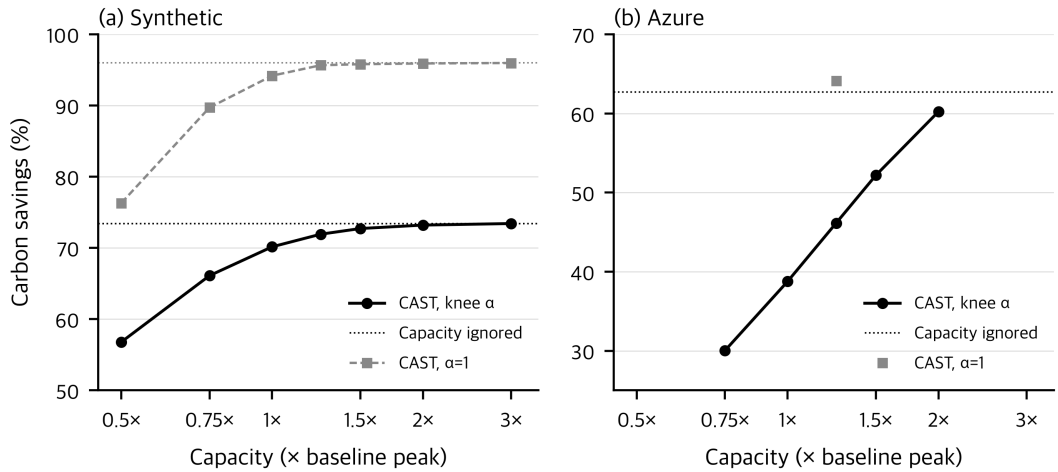


그림 10 상한에 따른 절감. (a) 합성, (*b*) Azure. 가로는 상한을 각 워크로드의 기준 피크(13, 34)의 배수로 적은 것이며, 계열 이름은 표 4와 같다. Azure의 α=1은 대표 상한 한 점이다.

Fig. 10 Savings against the capacity ceiling. (a) Synthetic, (b) Azure. The x axis is the ceiling as a multiple of each workload's baseline peak (13 and 34); series names follow Table 4. For Azure, α=1 is a single point at the representative ceiling.

같은 α에서 결합과 분리형의 차는 상한을 조일수록 벌어진다. α=1에서 CAST와 분리형의 차는 대표 상한 17에서 0.32%포인트(표 5)이고, 13에서 0.89%포인트, 10에서 1.77%포인트, 7에서 3.04%포인트다. 리전을 도착 때 고정하면 그 리전의 자리가 찼을 때 다른 리전으로 비켜 갈 수 없기 때문으로 읽힌다. 무릎점끼리는 두 설계가 고르는 α가 달라 같은 α의 비교가 되지 않으므로 여기서 견주지 않는다.

리전마다 상한을 달리 두어도 결론은 같다. 총합을 113~114로 같게 두고 세 가지로 나누었다. 리전마다 자기 기준 피크의 1.25배를 주면 70.04%, 깨끗한 세 리전(캘리포니아·프랑스·뉴욕)을 크게 주면 73.05%, 작게 주면 63.75%이고 모두 위반이 없다. 총합이 136인 균일 상한 17(71.93%)보다 총합이 작은 '크게' 배치가 더 줄이므로, 자리를 어디에 두느냐가 자리의 총량보다 크다. 같은 세 배치로 용량 무시의 배정을 채점하면 상한을 넘긴 시간은 '작게'에서 프랑스 57.4%·캘리포니아 40.2%, '크게'에서 캘리포니아 44.4%·프랑스 32.3%·일본 20.8%로, 어느 배치에서도 두 리전이 앞에 선다. 시간 이동의 이득이 어느 리전에서 나는지는 워크로드와 탄소 자료만의 성질이라 상한 배치와 무관하므로, 4.6절의 두 목록이 캘리포니아에서만 겹친다는 결과는 상한 배치를 바꿔도 유지된다.

워크로드 추첨에 따른 변동은 합성 워크로드를 시드 넷으로 다시 뽑아 잰다. 시드마다 기준 피크가 달라 상한과 예약도 같은 규칙으로 다시 정했으므로(상한 14~17, 예약 2~3), 아래 폭에는 상한의 변동도 들어 있다. 주 실험을 포함한 다섯 벌에서 CAST 무릎점은 71.51 ± 0.43%, α=1은 95.40 ± 0.28%, 분리형은 49.01 ± 0.28%, 공간만은 61.20 ± 0.22%, 시간만은 18.72 ± 0.05%, 용량 무시는 73.34 ± 0.27%이고(평균 ± 표준편차), 용량을 보는 모든 실행에서 위반은 0이다. 설계 사이의 차는 CAST와 분리형이 22.50 ± 0.66%포인트, CAST와 공간만이 10.32 ± 0.34%포인트로 시드 폭의 25배를 넘으므로 순위는 바뀌지 않는다.

예측 지평을 6, 12, 24시간으로 바꾸면 무릎점 절감은 71.63%, 71.04%, 71.93%로 0.9%포인트 안에서 움직이고 위반은 모두 0이다. 지평이 짧을수록 미룰 수 있는 범위가 좁아져 평균 미룸이 0.89, 1.25, 1.47시간으로 짧아진다.

네트워크 지연에 상한을 두면 결과가 크게 바뀐다. 식 (5)의 지연 상한을 모든 작업에 50, 100, 150 ms로 두면 무릎점 절감은 28.41%, 32.98%, 66.37%, α=1 절감은 38.09%, 46.83%, 89.85%로, 상한이 없을 때(71.93%, 95.65%)보다 낮고 위반은 모두 0이다. 50 ms와 100 ms에서는 한국, 일본, 인도에서 출발한 작업이(캘리포니아까지 109~134 ms, 프랑스까지 137 ms 이상) 가장 깨끗한 캘리포니아와 프랑스에 닿지 못해 홈에 남는 작업이 84.2%와 78.2%에 이르고, 절감이 상한이 없을 때의 절반 아래로 꺾인다. 4.1절의 지연 예산이 모든 작업에 공간 이동을 열어 준 것이 이만큼 공간 축에 유리하게 작용했다.

미룰 수 있는 작업의 비율 φ를 20%에서 60%까지 바꾸어도 결론은 그대로다. 등급 안의 비율은 유지하고, 상한과 예약은 각 φ의 기준 피크에서 같은 규칙으로 다시 정했다. CAST 무릎점은 φ 20, 30, 40, 50, 60%에서 72.30%, 72.54%, 71.93%, 71.45%, 70.01%, 분리형은 47.97~49.98%이고, 두 설계 모두 모든 φ에서 위반이 없다. 미룰 수 있는 작업이 늘수록 용량 무시의 위반은 각 φ의 상한으로 세어 15,233건에서 46,994건으로 세 배가 되고, 용량을 지키는 비용은 1.10~3.09%포인트로 φ=60%에서 가장 크다. 미룰 수 있는 작업이 많을수록 같은 골로 더 몰린다.

Azure 워크로드에서도 용량을 보는 모든 실행이 위반 없이 돈다. CAST의 무릎점과 α=1, 세 예측기, 분리형, 탐욕, 공간만과 시간만이 모두 위반 0이다. 용량 행을 지운 두 실행만 위반을 내며, 결합형은 61,400건과 초과 6,000시간에 최대 동시 실행 107건으로 상한 43의 2.5배에 이르고 분리형은 30,599건이다.

결과의 모양은 합성과 다르다. 작업이 길고 미룰 수 없는 작업시간이 51%라 시간 이동은 거의 값어치가 없어, 시간만은 1.53%를 줄이고 CAST의 무릎점(46.14%)도 공간만(45.71%)보다 0.43%포인트 앞설 뿐이다. 그래서 CAST와 분리형의 무릎점 차도 1.13%포인트로 작고, 트레이스를 다시 뽑은 두 시드에서는 2.02%포인트와 −0.95%포인트로 부호가 바뀐다. 합성 워크로드에서 본 결합형의 몫(표 5)은 Azure에서는 성립하지 않으며, 탄소만 보는 α=1에서는 분리형이 65.35%로 CAST(64.09%)보다 1.26%포인트 앞선다. 반대로 용량을 지키는 비용, 곧 용량 무시(62.70%)와의 차는 16.56%포인트로 합성의 1.47%포인트보다 열 배 넘게 크다. 무릎점이 예측기에 따라 뒤집히는 일도 없어 완전예지 46.55%, LSTM 46.14%, 24시간 전 값 45.74%로 0.4%포인트 안에 있다. 탄소만 보는 α=1은 64.09%다. 두 시드는 상한과 예약을 같은 규칙으로 다시 정해 47과 14, 39와 15가 되며, CAST는 49.59%와 41.71%, α=1은 67.54%와 59.57%를 낸다. 어느 축이 값어치를 내는지는 워크로드가 정한다. 본 연구의 두 워크로드에서 짧고 미룰 수 있는 작업이 많으면 시간 이동과 결합 설계가 절감을 더했고, 길고 미룰 수 없는 작업이 많으면 공간 이동과 용량이 결과의 거의 전부였다.

상한과 예약에 대한 반응도 다르다. 상한을 기준 피크 34의 0.75배, 1배, 1.25배, 1.5배, 2배로 바꾸면 무릎점

절감은 30.03%, 38.76%, 46.14%, 52.17%, 60.22%로 2배에서도 멈추지 않고, 용량 무시까지 2.48%포인트가 남는다(그림 10(*b*)). 예약 ρ를 6, 13, 19, 25로 바꾸면 절감은 50.44%, 46.14%, 42.49%, 39.88%로 준다. 합성에서 ρ는 상한 17의 35% 이하였지만 Azure에서는 상한 43의 14~58%라, 예약이 미래 슬롯의 자리를 직접 깎는다. 예약을 0으로 두어도 위반은 없고 절감은 51.35%로 ρ=13보다 5.20%포인트 높아, Azure에서 예약의 비용은 합성(0.13%포인트)보다 훨씬 크다. 대신 마감 직전에 시작이 강제되는 작업이 10,815건으로 ρ=13일 때(5,491건)의 두 배 가까이 된다. 도착 때 한 번 정하고 다시 풀지 않는 대조군은 39.97%로 위반이 없고, CAST가 6.17%포인트 더 줄여 재풀이의 몫은 Azure에서도 크다.

4.8 선행 정책 재현 비교

CICS, CASPER, Sukprasert, Caspian의 정책을 원문의 수식과 공개 코드 그대로 본 연구의 자료 위에 다시 구현하고 같은 회계(식 (1), 3.5절)로 채점하였다. CASPER는 탄소와 켜 둘 서버 대수의 가중합을 최소화하며 서버 대수를 함께 정하는 정수 계획으로, 원문 코드의 가중치 0.9와 서버 용량(서버당 10건, 리전당 100대)을 그대로 두고 매 시간 제출된 작업 수를 요청 수로 넘겼다. 그래서 CASPER는 우리 상한 17이 아니라 스스로 정한 서버 용량(시간당 8,000건이라 사실상 묶이지 않는다) 안에서 배정하며, 시각은 옮기지 않는다. 표 7의 필요 용량은 CASPER가 고른 배정에서 우리가 나중에 잰 최대 동시 실행 수다.

CICS는 리전마다 하루 전에 선형 계획 하나를 풀어, 그날 도착한 유연 작업의 총량은 그대로 두고 시간별 시작 허용량을 탄소집약도가 낮은 시각으로 옮긴 자원 곡선을 만든다. 원문의 CPU 사용량 대신 작업 건수를 쓰고, 시간별 허용량은 하루 평균의 2배까지로 묶었다. 첨두 전력 항의 가중치는 원문에 값이 없어 0으로 두어 탄소만 보게 했고, 그날의 탄소집약도는 실측값을 미리 안다고 두었다. 원문에는 작업별 마감이 없으므로, 대기 작업을 마감이 이른 순으로 허용량만큼 내보내고 마지막 기회가 온 작업은 허용량을 넘어도 내보낸다. 리전은 옮기지 않는다.

Sukprasert는 공개 코드의 탐색을 그대로 따른다. 제출부터 마감까지의 창 안에서, 작업 길이만큼 이어진 구간의 실측 탄소집약도 합이 가장 작은 시작 시각을 1시간 간격으로 고른다. 원문처럼 1년치 탄소집약도를 미리 알고 용량은 보지 않는다. 원문 실험은 여유를 24시간 이상의 고정값으로 쓸어 보지만, 여기서는 작업마다 자기 마감을 여유로 넣었다. 시간 이동은 홈 리전 위와 CAST가 정한 리전 위에서 두 번 돌리고, 공간+시간은 여덟 리전의 (리전, 시각) 짝 전체에서 같은 탐색을 한다.

Caspian은 공개 코드가 없어 논문의 정식화와 알고리즘 셋을 그대로 옮겼다. 매 1시간 24슬롯 지평의 선형 계획 완화를 풀고, 해의 값이 큰 작업부터 결합 목적의 계수가 큰 (리전, 슬롯)에 용량이 허락하는 대로 배정하며, 지금 슬롯에 시작하는 배정만 확정한다. 용량은 리전당 17의 하드 제약이고, 탄소는 LSTM 예측을 쓰며, 세 목적의 가중치와 리전별 전력 효율 계수는 모두 1로 두었다. 원문은 실행 시간을 정수 슬롯으로 세므로 1시간 미만 작업도 한 슬롯을 통째로 차지한다. 이 부풀림을 덜려고 슬롯 안에서 쓰는 자원량을 실행 시간(최대 1)으로 둔 판을 썼다. 원문의 마감은 지키지 않아도 되는 목표라서, 마감을 1시간 넘기도록 배정받지 못한 작업은 그 시각 홈 리전에서 바로 시작하게 했다. 리전, 기간, 워크로드가 원 논문과 달라 원 논문의 수치를 그대로 재현하는 것은 확인하지 못했다.

비교의 자는 둘이다. 하나는 상한 17에서의 위반 건수이고, 다른 하나는 그 정책의 배정이 위반 없이 돌려면 필요한 리전 상한, 곧 필요 용량이다. 상한 17은 본 연구가 고른 값이므로, 선행 정책이 그 상한을 넘겼다는 사실만으로는 정책의 좋고 나쁨을 가릴 수 없다. 필요 용량은 우리가 정한 상한과 무관하게 각 정책이 스스로 요구한 값이며, 1초짜리 순간 피크의 영향을 가리기 위해 시간 가중 99.9 백분위수를 함께 적는다. 결과는 표 7에, 절감과 필요 용량을 두 축에 놓은 것은 그림 11에 있다.

표 7 선행 정책을 같은 회계로 재현한 결과. 위반은 상한 17을 넘긴 배정 건수, 마감은 마감 위반 건수이고, 필요 용량은 리전별 최대 동시 실행의 최댓값(괄호는 시간 가중 99.9 백분위수)이며 기준 배정의 피크는 13이다. Suk.는 Sukprasert 등[6]의 정책으로, (home)은 홈 리전 위에서, (ours)는 CAST(무릎점)가 정한 리전 위에서 시각만 옮긴 것이다. CICS는 하루 전 자원 곡선, CASPER는 CAP 정책, Caspian은 슬롯 안 자원량을 실행 시간으로 둔 판이다.

Table 7 Prior policies reproduced under the same accounting. Viol. counts assignments past the ceiling of 17 and Deadl. deadline misses; Req. cap. is the largest per-region peak concurrency (time-weighted 99.9th percentile in parentheses); the baseline peak is 13. Suk. is the policy of Sukprasert et al. [6]: (home) shifts time on the home region and (ours) on the regions chosen by CAST at the knee. CICS uses the day-ahead capacity curve, CASPER its CAP policy, and Caspian the variant whose per-slot resource equals the run time.

Policy	Savings (%)	Viol.	Req. cap.	Deadl.	Latency (ms)
Baseline	—	0	13 (8)	0	0
CICS	13.89	0	12 (9)	0	0
Suk. temporal (home)	22.22	3,180	27 (19)	0	0
Suk. temporal (ours)	73.23	37,236	82 (64)	0	47.8
CASPER	95.30	11,061	29 (24)	0	120.4
Suk. spatial+temporal	96.63	61,539	129 (107)	0	120.4
Caspian	67.54	1,538	34 (28)	5,582	115.7
CAST, knee	71.93	0	17 (17)	0	47.8
CAST, α=1	95.65	0	17 (17)	0	120.0

절감률만 보면 CASPER(95.30%)와 Sukprasert 공간+시간(96.63%)이 CAST의 무릎점(71.93%)을 앞선다. 뒤엣것은 1년치 탄소집약도를 모두 미리 안다고 가정한 정책이다. 그러나 두 정책의 배치가 위반 없이 돌려면 리전 하나가 동시에 29건과 129건을 돌려야 한다. 기준 배정의 피크 13의 2.2배와 9.9배이며, 그 워크로드가 홈에서 한 번도 요구한 적 없는 동시 실행 수다. 지연까지 같은 짝으로 좁히면 차이가 분명하다. 둘 다 지연을 보지 않아 평균 지연이 120.4 ms와 120.0 ms로 거의 같은 CASPER와 CAST(α=1)는 절감도 95.30%와 95.65%로 비슷하지만, CAST(α=1)는 상한 17 안에서 이 절감을 낸다.

같은 축으로 좁혀 보아도 마찬가지다. 홈을 고정하고 시각만 옮기는 조건에서 CICS는 13.89%를 내며 필요 용량이 12로 기준보다도 낮다. 하루 전에 정한 자원 곡선 하나로 다음 날의 부하를 묶어 피크를 깎기 때문이며, 그 대신 개별 작업의 마감에 맞춰 시각을 고르지 않는다. Sukprasert의 시간 이동은 같은 조건에서 22.22%를 내지만 필요 용량이 27로 기준 피크의 2.1배다. 본 연구의 시간만은 18.78%로 둘 사이에 있고 최대 동시 실행이 상한 17을 넘지 않는다(표 4).

Sukprasert의 시간 이동은 어느 리전 위에서 돌리느냐에 따라 두 행으로 갈린다. 홈 리전 위에서는 22.22%이지만, CAST(무릎점)가 정한 리전 위에서 시각만 옮기면 73.23%가 되어 CAST보다도 조금 더 줄인다. 그 차 51.01%포인트는 시간 이동이 아니라 공간 이동의 몫이다. 그러나 이때 필요 용량은 82건으로 기준 피크의 6.3배이고 위반은 37,236건이다. CAST가 상한까지 채워 둔 리전 위에서 Sukprasert의 시간 이동이 작업을 다시 같은 골로 몰기 때문이다.

Caspian은 용량 17을 자기 슬롯 단위의 하드 제약으로 두고 마감을 양보한다. 절감은 67.54%이고, 마감을 넘긴 작업이 5,582건(3.90%)이다. 1시간 미만 작업을 실행 시간만큼의 분수로 세므로 자기 단위로는 상한을 지키지만, 시작 순간 단위로 다시 세면 1,538건이 상한을 넘고 필요 용량은 34로 기준 피크의 2.6배다. 평균 지연이 비슷한 CAST α=1(120.0 ms 대 115.7 ms)은 마감과 상한을 모두 지키며 28.11%포인트 더 줄이고, CAST 무릎점도 4.39%포인트 더 줄인다. 원 논문은 k3d로 만든 클러스터넷에 24시간 동안 작업 200건을 도착시켜 40시간을 평가했다[8].

순간 피크를 가린 99.9 백분위수로 다시 세어도 순서는 같다. CICS 9, Sukprasert 시간(홈) 19, CASPER 24, Caspian 28, Sukprasert 시간(본 연구 리전) 64, Sukprasert 공간+시간 107이다. CAST는 두 가중치 모두 17로, 순간 피크와 99.9 백분위수가 같다. 상한까지 채운 시간이 짧은 순간이 아니라는 뜻이다. Caspian을 빼면 모든 정책에서 마감을 넘긴 작업은 없었다.

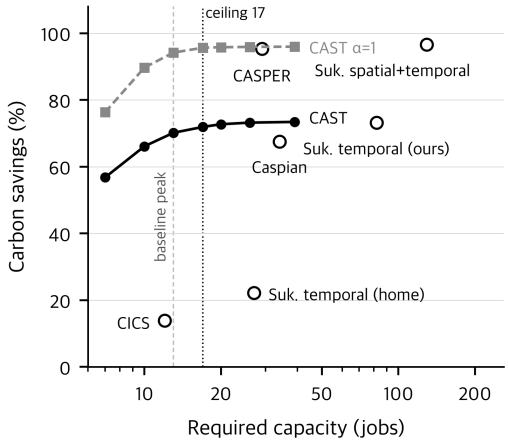


그림 11 선행 정책 재현 결과 비교(표 7). ○는 선행 정책, ●와 ■는 상한을 기준 피크의 0.5~3배로 바꾼 CAST의 무릎점과 α=1을 이은 것이고, 회색 점선은 기준 피크 13, 검은 점선은 상한 17이며 가로축은 로그 눈금이다.

Fig. 11 Comparison of reproduced prior policies (Table 7). Hollow circles are prior policies; filled circles and squares connect CAST at the knee and at α=1 with the ceiling swept from 0.5× to 3× the baseline peak; the gray dashed line marks the baseline peak of 13 and the black dashed line the ceiling of 17; the x axis is logarithmic.

4.9 계산 비용

CAST는 무릎점을 고르려고 매 슬롯 선형 계획을 열한 번 푼다. 1년 8,760 슬롯 동안 96,360번이며, 분수해가 나와 정수 계획으로 다시 푼 슬롯은 341개(3.9%)였다. 한 슬롯의 문제가 가장 컸을 때 변수는 7,848개였고, 정수 계획의 상대 최적성 허용오차는 HiGHS 기본값인 0.01%다. 무릎점 탐색을 포함한 슬롯당 풀이 시간은 평균 223 ms, 95 백분위수 380 ms, 최대 1,223 ms로 슬롯 길이 1시간에 비하면 무시할 만하며, 자료 적재와 채점까지 포함해 1년 전체를 4.1절의 기계 한 대에서 34.4분에 돌렸다. α 후보를 0.1 간격으로 둔 것은 계산과의 맞바꿈이다. 평가 구간 첫머리의 한 달(채점 작업 12,000건)에서 간격을 0.05로 좁혀 스물한 개를 풀면 절감은 66.19%에서 66.30%로 그 달 기준 배출의 0.11%포인트, 평균 지연은 1.1 ms 달라지지만 풀이 시간은 6.4분에서 12.4분으로 두 배가 된다.

α=1은 슬롯마다 한 번만 풀지만 정수 계획이 필요한 슬롯이 1,421개(16.2%)로 늘고 최대 변수가 9,488개라 1년에 14.4분이 걸린다. 분리형은 13.6분, 시간만은 7.7분, 공간만은 3.5분이다. 선형 계획 없이 작업마다 차례로 자리를 잡는 탐욕은 같은 무릎점 규칙으로 100.3분이 걸려, CAST가 세 배 가까이 빠르다. 탐욕은 순수 파이썬 구현이고 CAST는 HiGHS 풀이기를 쓰므로 구현 언어의 차이도 섞여 있다. 선행 정책의 재현은 CASPER 322초, CICS 90초, Sukprasert 셋이 각각 7~25초다. 작업시간이 합성의 7.8배인 Azure 워크로드에서도 한 슬롯 문제의 최대 변수는 8,032개로 합성(7,848개)과 거의 같다. 변수 수가 작업 수가 아니라 대기열의 길이에 매이기 때문이다. 늘어난 것은 정수 계획으로 다시 푼 슬롯(712개, 8.1%)과 슬롯당 풀이 시간(평균 337 ms, 95 백분위수 616 ms, 최대 1,772 ms)이며, 1년 전체는 52.3분으로 합성의 1.5배다. α=1은 최대 변수 9,936개, 정수 계획 슬롯 1,588개(18.1%), 17.0분이다.

5. 결론 및 향후 연구

본 논문은 탄소 예측이 실측에 가까워질수록 미룰 수 있는 작업이 같은 깨끗한 시각과 리전으로 몰려 리전 용량을 넘기는 문제를 다루고, 그 상한이 걸리는 리전이 시간 이동이 값어치를 내는 리전과 다름을 보였으며, 용량을 이미 정한 배정 전부와 함께 보며 대기 중인 작업 전부를 매 슬롯 한 판에 다시 배정하는 탄소 인지 스케줄링 시스템 CAST를 제안하였다. 작업이 시작할 때만 용량을 확인하는 2단계 설계에서는 LSTM 예측을 완전예지로 바꾸면 상한 위반이 18건에서 152건으로 늘었지만, 이미 정한 배정과 함께 용량을 보면 몰린 작업이 상한에서 멈추고 나머지가 다음으로 깨끗한 자리로 가서 예약 자리를 두지 않아도 위반이 없었다. 이를 위해 CAST는 용량을 입장 때 거르는 조건이 아니라 대기 중인 모든 작업이 함께 보는 제약으로 두고, 매 슬롯 아직 시작하지 않은 작업 전부의 실행 리전과 시작 슬롯을 선형 계획으로 다시 정하며, 리전 용량의 초과를 목적의 첫 항으로 최소화한다. 이 가운데 위반을 없애는 것은 함께 보는 제약이고, 매 슬롯 다시 푸는 것은 다시 풀지 않는 대조군보다 절감을 8.07%포인트 더한다. 여덟 리전의 1년치 실측 탄소집약도 위에서 배출량을 무릎점에서 71.93%, 지연을 보지 않는 α=1에서 95.65% 줄였고, 작업 143,200건 전량을 마감 안에 처리하면서 상한을 넘긴 배정은 없었다. 작업이 길고 작업시간의 절반이 미룰 수 없는 Azure VM 트레이스에서도 위반은 없었고, 용량을 지키는 비용은 16.56%포인트로 합성 워크로드의 열 배를 넘었다. 탄소만 보는 α=1에서 완전예지로 한 주를 한 번에 푼 하한과 견주면, 온라인으로 슬롯마다 다시 푸는 대가는 많아야 0.26%포인트였다. 탄소와 지연의 가중치는 슬롯마다 파레토 무릎점에서 자동으로 골랐으며, 무릎점은 사람이 α를 고르지 않아도 합성 워크로드에서 연간 전선 바로 안쪽에 머물렀고 Azure 워크로드에서는 같은 지연의

고정 α보다 1.89%포인트 더 줄였으며, 여섯 조건 모두에서 같은 지연의 고정 α보다 작업을 덜 기다리게 했다.

본 연구의 주된 발견은 용량이 결과를 바꾸는 방식에 있다. 리전을 먼저 정하고 시각을 나중에 고르며 용량을 입장 때만 확인하는 설계에서는 같은 리전의 작업들이 같은 예측 최저 시각으로 모이고, 예측 곡선의 골이 뾰족할수록 더 모여 상한 위반이 LSTM의 18건에서 24시간 전 값의 125건, 완전예지의 152건으로 는다. 용량을 모든 대기 작업이 함께 보는 제약으로 두면 어떤 예측기에서도 위반이 없다. 다만 공유 제약은 몰림을 지우지는 않고 해롭지 않게 만든다. 완전예지 예측에 LSTM 실행과 같은 α를 슬롯마다 주면, 탄소 최저 시각의 동시 실행은 평균 6.67건으로 LSTM 실행의 4.68건보다 많지만 절감은 71.33%로 LSTM의 71.93%와 0.59%포인트 차이에 그친다. 몰린 작업은 골을 상한까지 채우는 데서 멈추고, 위반으로도 손실로도 이어지지 않는다.

상한이 걸리는 자리는 고르지 않다. 용량 행을 지운 CAST에서는 상한을 넘긴 시간의 93.3%가 공간 이동이 모이는 캘리포니아와 프랑스에서 나고, 총합이 같은 세 가지 리전별 상한 배치에서도 이 두 리전이 앞에 서며, 캘리포니아에서는 그 74.2%가 현지 9시에서 13시 사이에 몰린다. 반면 시간 이동의 절감은 인도, 일본, 캘리포니아, 독일이 80.6%를 낸다. 상한이 걸리는 곳은 시간 이동이 값어치를 만드는 곳이 아니라 공간 이동이 모이는 곳이며, 둘이 겹치는 리전은 캘리포니아 하나다.

이 결과가 실무에 주는 함의는 세 가지다. 첫째, 탄소 인지 스케줄링의 이득은 그 배치가 요구하는 용량과 함께 보고해야 비교할 수 있다. Sukprasert 공간+시간 정책의 96.63%는 리전 하나가 동시에 129건을 돌려야 성립하는 숫자로, 기준 피크 13의 9.9배다. CASPER의 95.30%도 기준 피크의 2.2배인 29건을 요구한다.

둘째, 절대 배출량을 줄이는 것이 목표라면 시간 이동을 도입할지는 하루 안의 상대 변동이 아니라 절대 변동, 곧 날마다 최댓값과 최솟값의 차로 판단해야 한다. 절대 변동이 가장 큰 다섯 리전이 시간 이동 절감의 92.9%를 낸다. 프랑스는 상대 변동이 0.97로 큰 편이지만 절대 변동이 11.8 gCO_2/kWh로 가장 작아 0.3%만 낸다. 어느 축이 값어치를 내는지는 워크로드가 정한다. 본 연구의 두 워크로드에서 짧고 미룰 수 있는 작업이 많으면 시간 이동과 결합 설계가 절감을 더했고, 길고 미룰 수 없는 작업이 많으면 공간 이동과 용량이 결과의 거의 전부였다.

셋째, 용량을 입장 때만 보는 설계와 무릎점 규칙에서는 예측을 더 잘한다고 결과가 좋아지는 것은 아니다. 용량을 입장 때만 확인하는 설계에서는 골이 뾰족한 예측일수록 위반이 늘어, LSTM의 18건이 완전예지에서 152건이 된다. 탄소만 보는 α=1에서는 예측기를 바꿔도 절감이 0.6%포인트 안에서 움직이고, 완전예지와 LSTM의 차는 0.22%포인트, 용량을 지키는 비용은 0.31%포인트로 둘 다 작다. 그러나 용량을 보지 않은 배치는 프랑스 한 리전에 상한의 6.9배를 요구한다. 예측 오차보다 용량을 어디에 두느냐가 배치의 실현 가능성을 가른다.

한계는 다섯 가지다. 첫째, 탄소 회계가 평균 배출계수 위에 있다. 작업을 옮겨 실제로 바뀌는 한계 배출과 설비 효율(PUE)은 넣지 않았고, 리전 간 전송 에너지는 옮긴 작업마다 1 GB를 더해 보았을 때 절감이 1%포인트 안에서 움직이고 순서는 그대로였다(4.3절). 둘째, 실제 배포가 아닌 시뮬레이션이어서 클러스터 관리 계층과의 연동 비용을 확인하지 못했고, 슬롯 안에 도착하는 작업을 슬롯 시작에 안다고 두어 최대 1시간을 미리 본다. 셋째, 지연 등급은 두 워크로드 모두 모형값이다. Azure 워크로드도 도착과 실행 시간만 실측이고, 출발 리전은 균등하게 주었다. 이 지연 예산은 모든 작업에 공간 이동을 열어 두므로, 지연 상한을 100 ms로 두면 무릎점 절감이 32.98%로 꺾인다(4.7절).

넷째, 캘리포니아, 텍사스, 독일의 기상 입력이 예보가 아닌 실측값이어서 이 세 리전의 예측은 낙관적일 수 있다. 예측기를 기상 입력이 없는 24시간 전 값으로 바꾸면 α=1의 절감은 95.65%에서 95.59%로 0.06%포인트 움직이지만, 무릎점에서는 71.93%에서 64.44%로, 평균 지연은 47.8 ms에서 39.2 ms로 함께 움직인다. 무릎점 결과는 예측기에 매여 있으며, 그 차이의 대부분은 무릎점 규칙이 예측 곡선의 모양에 따라 다른 α를 고르는 데서 온다(4.5절). 무릎점은 선형 계획 완화의 좌표로 고르고 분수해가 난 슬롯만 그 α에서 정수 계획으로 다시 풀므로, 그 슬롯들에서는 정수해의 좌표로 고른 무릎점과 다를 수 있다. 다섯째, 탄소 자료가 2025년 한 해이고 예측 지평이 24시간이어서, 하루를 넘는 작업의 리전도 앞 24시간 예측으로 정한다. 같은 α에서 결합형과 분리형을 견준 비교는 주 실험의 시드 한 벌에서만 했다. 선행 정책은 원문 수식대로 다시 구현했으나 원 논문의 자료가 아니어서 원 수치의 재현은 확인하지 못했다.

향후 과제는 실제 예보 API 연동, 한계 배출계수를 쓰는 회계, 24시간을 넘는 예측 지평[29], 그리고 리전 상한을 수요와 설비에서 추정하는 방법이다. 무릎점이 연간 전선의 지연 쪽 안쪽에 놓이는 편향은 무릎점을 슬롯 전선이 아니라 그때까지 쌓인 누적 전선에서 고르면 줄 수 있으며, 이 또한 향후 연구로 남긴다. 마지막은 Zanotto 등이 남긴 과제이기도 하며[9], 본 연구가 상한을 실험 변수로 두고 쓸어 본 결과가 그 출발점이 될 수 있다.

용량을 지키면서 얻은 절감이 실제 클러스터에서도 유지되는지 검증할 계획이다.

참고문헌

[1] International Energy Agency, Energy and AI, World Energy Outlook Special Report, IEA, Paris, 2025.

[2] European Parliament and Council, Directive (EU) 2023/1791 of the European Parliament and of the Council of 13 September 2023 on Energy Efficiency (recast), Official Journal of the European Union, 2023.

[3] A. Radovanović, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D. Xiao, M. Haridasan, P. Hung, N. Care, S. Talukdar, E. Mullen, K. Smith, M. Cottman, and W. Cirne, “Carbon-Aware Computing for Datacenters,” IEEE Transactions on Power Systems, Vol. 38, No. 2, pp. 1270-1280, Mar. 2023.

[4] A. Souza, S. Jasoria, B. Chakrabarty, D. Irwin, P. Shenoy, A. Bridgwater, A. Lundberg, F. Skogh, and A. Ali-Eldin, “CASPER: Carbon-Aware Scheduling and Provisioning for Distributed Web Services,” Proc. of the 14th Int. Green and Sustainable Computing Conf. (IGSC), pp. 67-73, Oct. 2023.

[5] D. Maji, B. Pfaff, V. P R, R. Sreenivasan, V. Firoiu, S. Iyer, C. Josephson, Z. Pan, and R. K. Sitaraman, “Bringing Carbon Awareness to Multi-Cloud Application Delivery,” Proc. of the 2nd Workshop on Sustainable Computer Systems (HotCarbon), pp. 1-6, Jul. 2023.

[6] T. Sukprasert, A. Souza, N. Bashir, D. Irwin, and P. Shenoy, “On the Limitations of Carbon-Aware Temporal and Spatial Workload Shifting in the Cloud,” Proc. of the 19th European Conf. on Computer Systems (EuroSys), pp. 924-941, Apr. 2024.

[7] W. A. Hanafy, L. Wu, D. Irwin, and P. Shenoy, “CarbonFlex: Enabling Carbon-aware Provisioning and Scheduling for Cloud Clusters,” arXiv:2505.18357, 2025.

[8] T. Bahreini, A. N. Tantawi, and O. Tardieu, “Caspian: A Carbon-aware Workload Scheduler in Multi-Cluster Kubernetes Environments,” Proc. of the 32nd IEEE Int. Symp. on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), pp. 1-8, Oct. 2024.

[9] M. Zanotto, L. Vicentini, R. Vreto, F. Lumpp, D. Braga, and S. Fiore, “A User-centric Kubernetes-based Architecture for Green Cloud Computing,” arXiv:2509.13325, 2025.

[10] W. Feng, A. Dasgupta, Z. D. Tekler, S. Ahuja, J. Zheng, and X. Jiang, “COAST: Congestion-Aware Start-Time Recommendations for Carbon-Aware HPC Jobs,” arXiv:2609.05443, 2026.

[11] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, and L. Thamsen, “Let's Wait Awhile: How Temporal Workload Shifting Can Reduce Carbon Emissions in the Cloud,” Proc. of the 22nd Int. Middleware Conf. (Middleware), pp. 260-272, Dec. 2021.

[12] W. A. Hanafy, Q. Liang, N. Bashir, D. Irwin, and P. Shenoy, “CarbonScaler: Leveraging Cloud Workload Elasticity for Optimizing Carbon-Efficiency,” Proc. of the ACM on Measurement and Analysis of Computing Systems (POMACS), Vol. 7, No. 3, Article 57, Dec. 2023.

[13] W. A. Hanafy, Q. Liang, N. Bashir, A. Souza, D. Irwin, and P. Shenoy, “Going Green for Less Green: Optimizing the Cost of Reducing Cloud Carbon Emissions,” Proc. of the 29th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Vol. 3, pp. 479-496, 2024.

[14] K. Xu, D. Sun, H. Tian, J. Zhang, and K. Chen, “GREEN: Carbon-efficient Resource Scheduling for Machine Learning Clusters,” Proc. of the 22nd USENIX Symp. on Networked Systems Design and Implementation (NSDI), pp. 999-1014, Apr. 2025.

[15] G. Attenni, Y. Moawad, N. Bartolini, and L. Thamsen, “Spatio-Temporal Shifting to Reduce Carbon, Water, and Land-Use Footprints of Cloud Workloads,” arXiv:2512.08725, 2025.

[16] A. Lechowicz, N. Christianson, B. Sun, N. Bashir, M. Hajiesmaili, A. Wierman, and P. Shenoy, “CarbonClipper: Optimal Algorithms for Carbon-Aware Spatiotemporal Workload Management,” Proc. of the ACM on Measurement and Analysis of Computing Systems (POMACS), Vol. 9, 2025.

[17] S. Hall, F. Micheli, G. Belgioioso, A. Radovanović, and F. Dörfler, “Carbon-Aware Computing for Data Centers with Probabilistic Performance Guarantees,” arXiv:2410.21510, 2024.

[18] D. Maji, R. K. Sitaraman, and P. Shenoy, “DACF: Day-Ahead Carbon Intensity Forecasting of Power Grids using Machine Learning,” Proc. of the 13th ACM Int. Conf. on Future Energy Systems (e-Energy), pp. 188-192, Jun.-Jul. 2022.

[19] A. Schrijver, Theory of Linear and Integer Programming, Wiley, Chichester, 1986.

[20] A. J. Hoffman and J. B. Kruskal, “Integral Boundary Points of Convex Polyhedra,” in H. W. Kuhn and A. W. Tucker (Eds.), Linear Inequalities and Related Systems, Annals of Mathematics Studies 38, Princeton University Press, pp. 223-246, 1956.

[21] J. Branke, K. Deb, H. Dierolf, and M. Osswald, “Finding Knees in Multi-objective Optimization,” Proc. of the 8th Int. Conf. on Parallel Problem Solving from Nature (PPSN VIII), LNCS 3242, pp. 722-731, Sep. 2004.

[22] M. Zeleny, “Compromise Programming,” in J. L. Cochrane and M. Zeleny (Eds.), Multiple Criteria Decision Making, University of South Carolina Press, Columbia, pp. 262-301, 1973.

[23] Electricity Maps. API Documentation [Online]. Available: https://docs.electricitymaps.com (downloaded 2026, Oct. 12)

[24] Electricity Maps. Methodology [Online]. Available: https://www.electricitymaps.com/data/methodology (downloaded 2026, Sep. 22)

[25] Microsoft. Azure Network Round-Trip Latency Statistics [Online]. Available: https://learn.microsoft.com/en-us/azure/networking/azure-network-latency (downloaded 2026, Sep. 22)

[26] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms,” Proc. of the 26th ACM Symp. on Operating Systems Principles (SOSP), pp. 153-167, Oct. 2017.

[27] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, “Heterogeneity and Dynamicity of Clouds at Scale: Google Trace Analysis,” Proc. of the 3rd ACM Symp. on Cloud Computing (SoCC), Article 7, Oct. 2012.

[28] J. Aslan, K. Mayers, J. G. Koomey, and C. France, “Electricity Intensity of Internet Data Transmission: Untangling the Estimates,” Journal of Industrial Ecology, Vol. 22, No. 4, pp. 785-798, 2018.

[29] D. Maji, P. Shenoy, and R. K. Sitaraman, “CarbonCast: Multi-Day Forecasting of Grid Carbon Intensity,” Proc. of the 9th ACM Int. Conf. on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys), pp. 198-207, Nov. 2022.